

Seguridad Web

PRESENTADO POR
Michel Maes Bermejo

ENTIDAD
Universidad Rey Juan Carlos

MÓDULO
Avanzado

Presentación

Michel Maes Bermejo

- Profesor e Investigador de la Rey Juan Carlos desde hace 8 años
- Intereses: Desarrollo Web, Sistemas Distribuidos, Computación en la Nube ...
- Contacto: michel.maes@urjc.es



Seguridad Web

- Conceptos básicos de seguridad web
- OWASP TOP 10
- Otras vulnerabilidades
- Conclusiones

Seguridad Web

- **Conceptos básicos de seguridad web**
- OWASP TOP 10
- Otras vulnerabilidades
- Conclusiones

Conceptos básicos de seguridad web

- **Autenticación:** garantizar que una entidad es quien dice ser.
- **Autorización:** garantizar que un usuario autenticado esté autorizado para acceder a un determinado recurso antes de que pueda accederlo.
- **Integridad:** garantizar al receptor de un mensaje que éste coincide exactamente con el enviado por el emisor.
- **Confidencialidad:** garantiza que los datos no sean revelados a un usuario no autorizado para conocerlos.

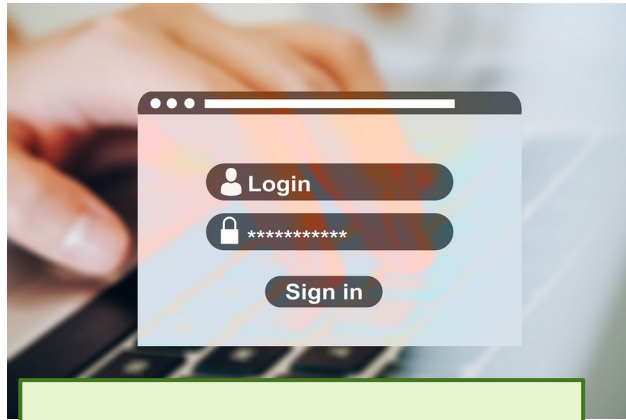
Conceptos básicos de seguridad web

Autenticación

- Es el proceso de **verificar la identidad de un usuario**, dispositivo o sistema.
 - ¿Eres quien dices ser?
- En la web, esto se traduce en demostrar que la persona que intenta acceder a una cuenta es realmente la dueña de esa cuenta.

Conceptos básicos de seguridad web

Autenticación



Algo que **sabes**



Algo que **tienes**



Algo que **eres**

Conceptos básicos de seguridad web

Autorización

- Es el proceso de verificar si un usuario autenticado tiene los privilegios necesarios para acceder a un recurso específico o realizar una acción concreta.
- La autenticación siempre debe ir antes que la autorización.
 - Ahora que sé quién eres, ¿qué tienes permiso para hacer?

Conceptos básicos de seguridad web

Autorización

- Para llevar a acabo el proceso de autorización, utilizamos diversas herramientas:
 - Permisos
 - Listas de control de acceso (ACLs)
 - Políticas
 - Otros: Roles, tokens ...

Conceptos básicos de seguridad web

Autorización

- Un **permiso** es la unidad más pequeña y atómica de la autorización.
- Es una acción específica sobre un recurso específico.



Fuente:

<https://www.redeszone.net/tutoriales/servidores/controlar-permisos-archivos-carpetas-servidor-linux/>

Conceptos básicos de seguridad web

Autorización

- Una **lista de control de acceso (ACL)** es una lista estática que está ligada a un recurso.
- Asigna directamente sujetos (usuarios, grupos) a permisos.

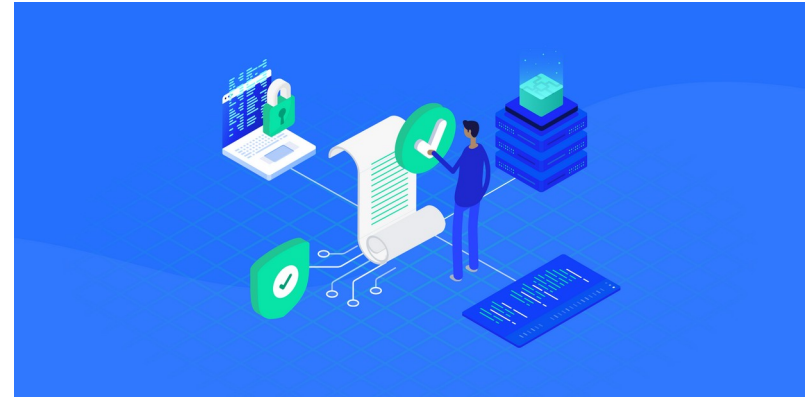
	Armas	El Anillo	Jamón	Diplomacia	Cerveza
Gandalf	Negar	Negar	Permitir	Permitir	Permitir
Aragorn	Permitir	Negar	Permitir	Permitir	Permitir
Bilbo	Negar	Negar	Negar	Negar	Permitir
Frodo	Negar	Permitir	Negar	Negar	Permitir
Gollum	Negar	Negar	Permitir	Negar	Negar
Legolas	Permitir	Negar	Permitir	Permitir	Permitir
Pippin	Negar	Negar	Negar	Permitir	Permitir
Merry	Negar	Negar	Negar	Negar	Permitir

Fuente: <https://www.securityartwork.es/2012/05/31/listas-de-control-de-acceso-acl/>

Conceptos básicos de seguridad web

Autorización

- Una **política** es un conjunto de reglas lógicas de alto nivel que evalúan el contexto antes de conceder un permiso.
- No están ligadas a un solo recurso, sino que gobiernan el sistema general.



Conceptos básicos de seguridad web

Autorización

Característica	Permiso	ACL	Política
Definición	Acción atómica permitida.	Lista de quién puede hacer qué en un recurso.	Reglas de negocio lógicas y condicionales.
Enfoque	La acción (Leer, Escribir).	El recurso (Archivo.txt).	El contexto (Rol, Tiempo, IP, Dispositivo).
Complejidad	Bajo	Medio	Alto
Ejemplo de código	<code>can_delete</code>	<pre>{'user1': ['read'], 'user2': ['read', 'write']}</pre>	<pre>if (user.role == admin && time < 17:00)</pre>

Conceptos básicos de seguridad web

Autorización

- Un **rol** es una capa intermedia entre el usuario (identidad) y un permiso (acción)
 - Sustituye al ACL → En lugar de revisar una lista, solo se comprueba el rol, que contiene una lista de permisos asociada

Conceptos básicos de seguridad web

Autorización

- Un **token** combina autenticación y autorización
- Proceso:
 - El usuario envía sus credenciales (user/pass)
 - El servidor las valida, obtiene el rol de ese usuario, empaqueta la información en un token, lo firma y se lo devuelve al usuario
 - El usuario usa ese token para realizar acciones

PROCESO DE AUTORIZACIÓN BASADA EN ROLES Y TOKENS (RBAC)



Conceptos básicos de seguridad web

Integridad

- Garantizar que los datos no han sido modificados, corrompidos o destruidos de forma no autorizada durante su transmisión o almacenamiento.
- Lo que se envió debe ser exactamente lo mismo que lo que se recibió.
 - ¿Ha sido alterada la información?
- Implica “firmar” los datos (funciones hash, firmas digitales ...)

Conceptos básicos de seguridad web

Confidencialidad

- Garantizar que los datos solo sean accesibles para las personas o sistemas autorizados.
- Si alguien intercepta la comunicación o roba la base de datos, no debería poder entender la información.
 - ¿Está la información protegida contra miradas indiscretas?
- Cifrado de comunicaciones (HTTPS) y de datos sensibles

Seguridad Web

- Conceptos básicos de seguridad web
- **OWASP TOP 10**
- Otras vulnerabilidades
- Conclusiones

OWASP TOP 10



TOP10



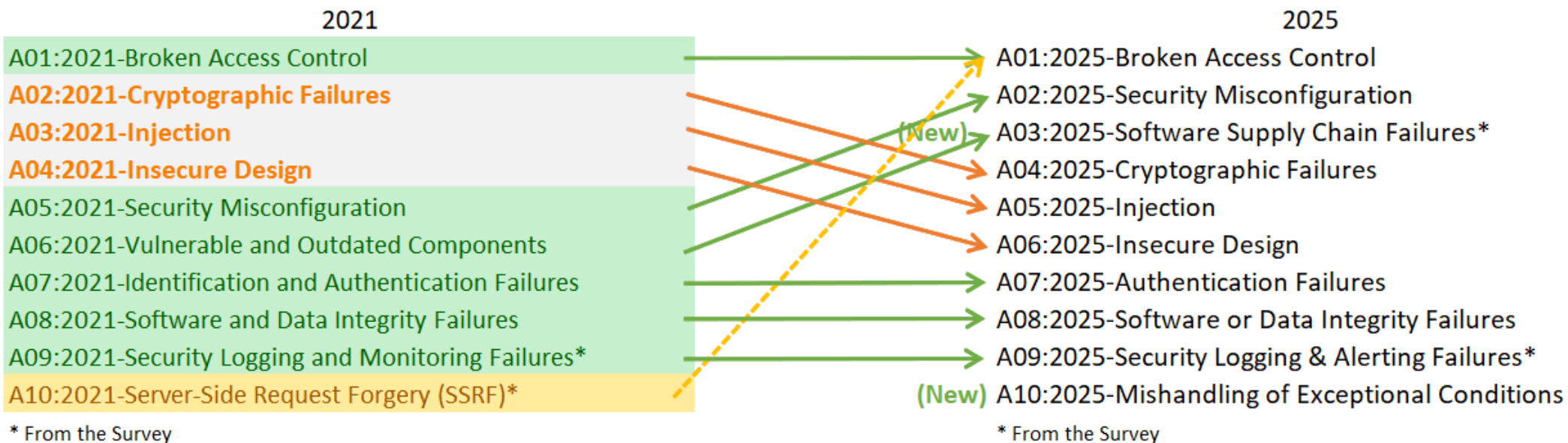
<https://owasp.org/Top10/2025/>

OWASP TOP 10

- Documento que recoge los **10 riesgos de seguridad más importantes en aplicaciones web**.
- Se publica y actualiza cada tres o cuatro años por parte de la fundación OWASP.
- Su objetivo es **concienciar acerca de la seguridad web** identificando algunos de los riesgos más críticos para las organizaciones.
- “Bebe” bastante del CWE de Mitre. <https://cwe.mitre.org/>

OWASP TOP 10

- Usaremos el informe más reciente (2025)



Fuente: <https://owasp.org/Top10/2025/>

OWASP TOP 10

- Abordaremos las vulnerabilidades más relevantes, indicando vulnerabilidades concretas y posibles contramedidas



Vulnerabilidad



Contramedida

A01:2025 Broken Access Control



- Los mecanismos de control de acceso no deberían permitir a un usuario **realizar acciones más allá de aquellas para las que tienen permisos** (en base a una política definida, un rol, etc.)
- Los fallos en estos mecanismos llevan al acceso, la modificación o la destrucción de datos por parte de un usuario no autorizado para ello.
- Actualmente ocupa el TOP 1 de OWASP (**vulnerabilidad más común**).

OWASP TOP 10

A01:2025 Broken Access Control



- Los fallos más frecuentes que causan esta vulnerabilidad son (1/2):
 - Violación del principio del mínimo privilegio o la denegación de acceso por defecto.
 - Bypass de los controles de acceso modificando la URL, el estado interno de la aplicación, el código fuente o las peticiones a la API.
 - Permitir ver o editar la cuenta de otro usuario proporcionando su identificador único (IDOR, *Insecure Direct Object References*)
 - Acceso a la API sin controles de acceso para los métodos POST, PUT y DELETE.

OWASP TOP 10

A01:2025 Broken Access Control



- Los fallos más frecuentes que causan esta vulnerabilidad son (2/2):
 - Elevación de privilegios: actuar como un usuario registrado sin estar identificado o actuar como un admin cuando se está identificado como un usuario.
 - Manipulación de metadatos: modificación de un JWT, modificación de cookies, modificación de campos ocultos.
 - CORS mal configurado que permita acceso a la API desde orígenes no autorizados o confiables.
 - Forzar el descubrimiento de páginas que requieren autenticación.

OWASP TOP 10

A01:2025 Broken Access Control



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/3):
 - **Denegar por defecto** el acceso a los recursos, salvo a los recursos públicos.
 - Implementar mecanismos de control de acceso **una sola vez y reutilizarlos** a lo largo de la aplicación).
 - Modelar el control de acceso, forzando el **registro de la propiedad** de un recurso, en lugar de permitir a un usuario acciones CRUD sobre cualquier registro.

A01:2025 Broken Access Control



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/3):
 - **Deshabilitar el listado de directorios** del servidor web y asegurar que los ficheros de metadatos (como .git) y los de backup no están presentes en la raíz de la web.
 - **Control de fallos de acceso** (incluyendo log) y con servicio de alerta al administrador cuando sea necesario.
 - **Limitar el nº de peticiones** que se permiten a la API y el acceso a los controladores para minimizar el uso de herramientas automatizadas.

A01:2025 Broken Access Control



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (3/3):
 - Los identificadores de sesión stateful se deben invalidar en el servidor después de hacer logout.
 - Los tokens sin estado (stateless, como JWT) deben tener un tiempo de vida corto, de forma que la ventana de oportunidad de un atacante se minimice.
 - Si tienen un tiempo de vida largo por necesidades de la aplicación, se recomienda seguir los estándares de Oauth para revocar los accesos.

A01:2025 Broken Access Control



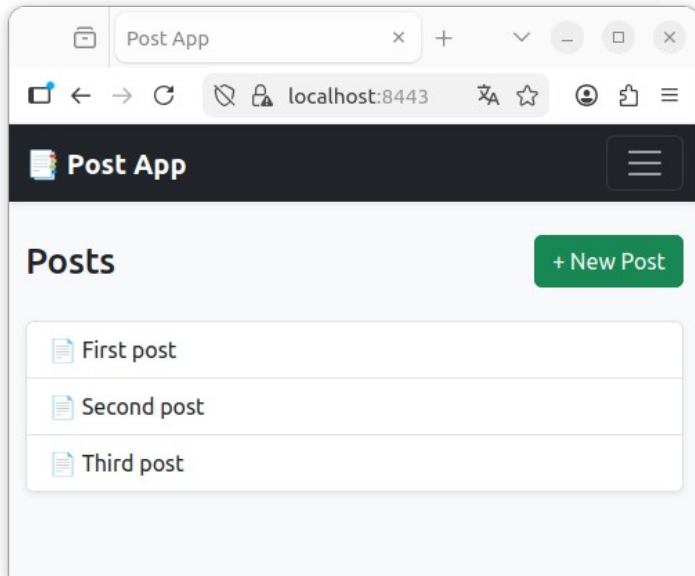
- Ejemplo – Aplicación Spring de Posts
 - Aplicación web con BBDD + Seguridad
 - Los usuarios pueden crear Posts que les pertenecen
 - Solo los dueños de un post (o el admin) deberían poder borrarlo o editarlo

OWASP TOP 10

A01:2025 Broken Access Control



- Ejemplo – Aplicación Spring de Posts



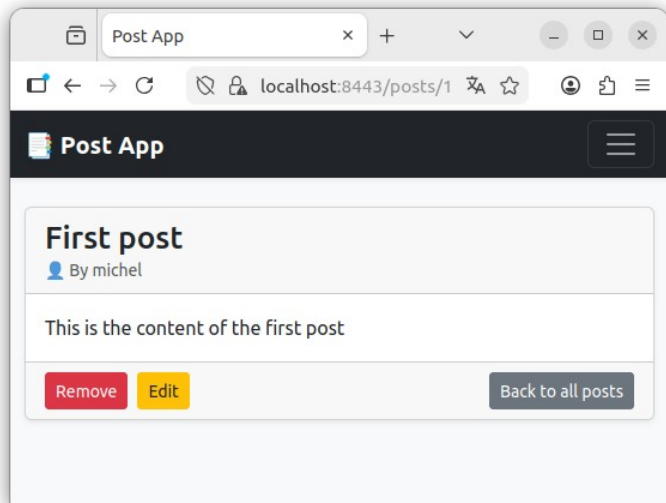
Al usuario “michel” le pertenecen los posts 1 y 3. Al usuario “oscar” le pertenece el post 2

OWASP TOP 10

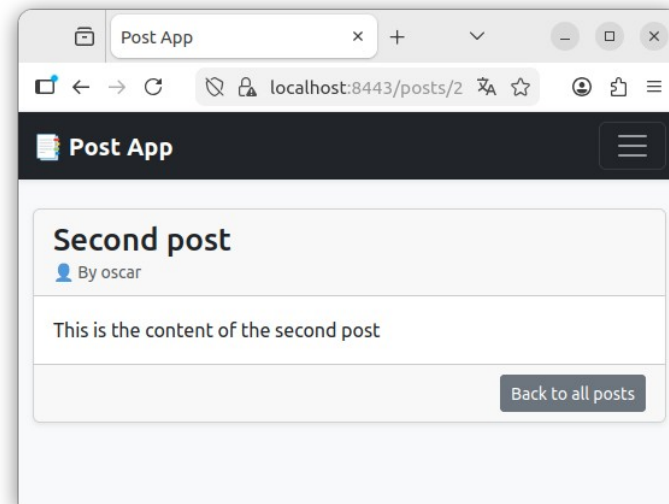
A01:2025 Broken Access Control



- Ejemplo – Aplicación Spring de Posts



El usuario "michel" tiene disponibles las acciones de borrar y editar en su post, pero no en el del usuario "oscar"

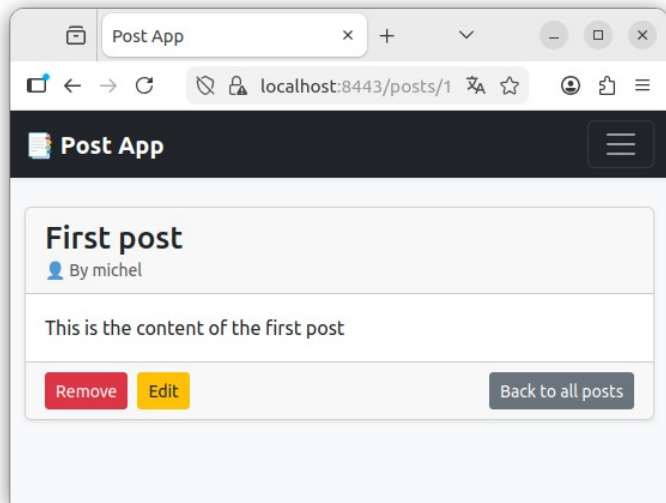


OWASP TOP 10

A01:2025 Broken Access Control



- Ejemplo – Aplicación Spring de Posts



security_ejem1_broken_access_control

```
<form class="d-inline" action="/removepost/1" method="post">  
  <button class="btn btn-danger btn-sm" type="submit">Remove</button>  
  <input type="hidden" name="_csrf" value="Do3dgin1X9jJ9F-wR0t7tkqKlMc">  
</form>
```

El usuario "michel" podría modificar el código HTML del botón "Remove" para que borrara el post 2 en lugar del 1 y conseguir borrarlo

```
<form class="d-inline" action="/removepost/2" method="post">  
  <button class="btn btn-danger btn-sm" type="submit">Remove</button>  
  <input type="hidden" name="_csrf" value="Do3dgin1X9jJ9F-wR0t7tkqKlMc">  
</form>
```

OWASP TOP 10

A01:2025 Broken Access Control

A screenshot of a web browser window. The browser's address bar shows the URL 'localhost:8443/editpost/2'. The page header for 'Post App' includes a user profile 'michel' and a 'Log Out' button. The main content is a modal form titled 'Edit Post "Second post"'. It contains two input fields: 'Title' with the value 'Second post' and 'Content' with the value 'This is the content of the second post'. At the bottom of the form are two buttons: 'Cancel' and 'Save Changes'.

El usuario "michel" navegar a la ruta `https://localhost:8443/editpost/2` para editar el post del usuario "oscar"

A01:2025 Broken Access Control



- Ejemplo – Aplicación Spring de Posts
 - En ambos casos estamos haciendo operaciones sobre recursos que no son nuestros
 - El problema ha sido confiar en la “ocultación” de los botones

A01:2025 Broken Access Control



- Ejemplo – Aplicación Spring de Posts
 - Para solucionar la vulnerabilidad, al hacer una operación sobre un recurso, es necesario realizar una comprobación **en el servidor**

A02:2025 - Security Misconfiguration



- Una mala configuración de seguridad se produce cuando un sistema, una aplicación o un servicio en la nube **se configura incorrectamente** desde el punto de vista de la seguridad, lo que crea vulnerabilidades.
 - **Omnipresencia:** Casi cualquier sistema requiere configuración. Si no se hace con seguridad en mente, es probable que haya fallos.
 - **Facilidad de explotación:** Muchos de estos fallos son triviales de encontrar para un atacante (ej. contraseñas por defecto, servicios innecesarios abiertos) y a menudo no requieren conocimientos técnicos avanzados.
 - **Impacto Potencial:** Una simple mala configuración puede dar acceso completo al servidor, permitir la filtración de datos sensibles, facilitar la escalada de privilegios o incluso la ejecución remota de código.

OWASP TOP 10

A02:2025 - Security Misconfiguration



- Los fallos más frecuentes que causan esta vulnerabilidad son (1/2):
 - **Credenciales por defecto:** Dejar usuarios y contraseñas de fábrica (ej. admin/password) en paneles de administración de bases de datos, routers, CMS, etc.
 - **Servicios/Funciones innecesarias:** Mantener activos servicios que la aplicación no necesita (ej. FTP, puertos abiertos, listado de directorios activo, páginas de ejemplo, depuración habilitada en producción)
 - **Mensajes de error verbosos:** Mostrar errores detallados del servidor (stack traces, rutas de archivos, versiones de software) directamente al usuario final. Esto proporciona información valiosa a los atacantes para planificar su próximo movimiento.

OWASP TOP 10

A02:2025 - Security Misconfiguration



- Los fallos más frecuentes que causan esta vulnerabilidad son (2/2):
 - **Sistemas desactualizados o sin parches:** No aplicar actualizaciones de seguridad críticas al sistema operativo, servidor web, servidor de aplicaciones o bibliotecas de terceros. Las vulnerabilidades conocidas son explotadas rápidamente.
 - **Configuraciones de seguridad en la nube incorrectas:** Permisos de almacenamiento (S3 buckets, Azure Blobs) abiertos al público, grupos de seguridad mal configurados que exponen puertos internos a internet, claves de API incrustadas en el código, etc.

OWASP TOP 10

A02:2025 - Security Misconfiguration



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/2):
 - **Automatización de la configuración:** Utilizar herramientas de gestión de configuración (ej. Ansible, Chef, Puppet, Terraform) para definir y desplegar configuraciones seguras y consistentes de forma automática.
 - **Imágenes base seguras:** Empezar con imágenes de servidor (Docker, VMs) endurecidas y minimalistas, que solo incluyan lo estrictamente necesario.
 - **Auditorías y escaneos regulares:** Implementar procesos automatizados para escanear y auditar continuamente las configuraciones de seguridad en busca de desviaciones de las mejores prácticas.

OWASP TOP 10

A02:2025 - Security Misconfiguration



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/2):
 - **Eliminar lo innecesario:** Seguir el principio de mínimo privilegio y superficie de ataque mínima. Desactivar o desinstalar cualquier servicio, función o componente innecesario.
 - **Gestión de parches efectiva:** Establecer un proceso rápido y sistemático para evaluar, probar y aplicar parches de seguridad críticos.
 - **Principio de menor privilegio:** Asegurarse de que los procesos, usuarios y cuentas de servicio tengan solo los permisos mínimos necesarios para realizar su función.

OWASP TOP 10

A02:2025 - Security Misconfiguration



- Ejemplo – Credenciales expuestas en Spring

```
#Mail Configuration
spring.mail.host=smtp.gmail.com
spring.mail.port=587
spring.mail.username=[REDACTED]@gmail.com
spring.mail.password=ubsd yxpr njja mujn
spring.mail.properties.mail.smtp.auth=true
spring.mail.properties.mail.smtp.starttls.enable=true
```

Archivo *application.properties* (Spring) en un repositorio de Github público con claves de producción expuestas (cuenta de correo)

OWASP TOP 10

A03:2025 - Software Supply Chain Failures



- Aborda los riesgos introducidos a través de componentes, librerías, herramientas de construcción externos
 - **Interconexión:** El software moderno depende en gran medida de código de terceros. Una sola dependencia vulnerable o maliciosa puede comprometer muchas aplicaciones (por ejemplo, Log4j).
 - **Confianza:** Asumir que todas las dependencias y herramientas son confiables es una configuración defectuosa conceptualmente.
 - **Escala:** Un solo fallo en la cadena de suministro puede tener un impacto masivo y generalizado

OWASP TOP 10

A03:2025 - Software Supply Chain Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son:
 - No llevar un control preciso de las versiones de los componentes (y sus dependencias transitivas)
 - El software de terceros es vulnerable, no cuenta con soporte o está desactualizado (SO, servidor web, sistema de gestión de BBDD)
 - No realizar escaneo de vulnerabilidades
 - Sin proceso de gestión de cambios: cualquiera puede llevar un cambio a producción
 - Uso de componentes de fuentes no fiables

OWASP TOP 10

A03:2025 - Software Supply Chain Failures



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en
 - Uso de repositorios verificados/privados
 - Implementación de Análisis de Composición de Software (SCA) para escanear dependencias
 - Aseguramiento de pipelines CI/CD (secretos encriptados, entornos aislados)
 - Verificación de la firma de código/integridad de artefactos
 - Eliminar librerías no utilizadas
 - Estar al día de los CWE y usar herramientas que nos notifiquen

A03:2025 - Software Supply Chain Failures



- **Ejemplo – Dependabot de GitHub**
 - GitHub (*dependabot*) analiza de manera estática el código subido revisando versiones de dependencias
 - Nos avisa si detecta alguna librería vulnerable:
 - <https://github.com/Maes95/bookmarks/security/dependabot/1>

OWASP TOP 10

A03:2025 - Software Supply Chain Failures



- Ejemplo – Maven Central
 - Maven Central en su repositorio de librerías para proyectos Java
 - Indica en qué versiones se ha detectado una vulnerabilidad

Latest Versions

117 versions →

Central (92)

Jahia (1)

Redhat GA (11)

Redhat EA (1)

Imcode (1)

+2 more

	VERSION	VULNERABILITIES	USAGES	DATE
8.0.x	8.0.33	1 vulnerability	437	Apr 18, 2023
	8.0.32	1 vulnerability	150	Jan 19, 2023
	8.0.31	1 vulnerability	190	Oct 18, 2022
6.0.x	6.0.6	4 vulnerabilities	246	Feb 23, 2017
	6.0.5	4 vulnerabilities	77	Oct 10, 2016
	6.0.4	4 vulnerabilities	27	Aug 12, 2016
	5.1.49	4 vulnerabilities	196	Apr 28, 2020

OWASP TOP 10

A03:2025 - Software Supply Chain Failures



- **Ejemplo Axios:** En marzo de 2026 se produjo un supply chain attack que comprometió una popular librería para realizar peticiones HTTP desde JS (Axios)
 - **Método:** Se introdujo una librería maliciosa como dependencia (*plain-crypto-js@4.2.1*) en la versión de axios 1.14.1
 - **Payload:** Esta librería descargaba un troyano remoto
 - **Scope:** La versión vulnerable estuvo presente 3 h (la librería es descargada 100M de veces a la semana)
 - **Objetivo:** Ganar acceso a pipelines de CI/CD, obtener secretos y credenciales

OWASP TOP 10

A03:2025 - Software Supply Chain Failures



- **Ejemplo Axios (Contramedidas)**
 - Realizar un downgrade de la versión (1.14.1 → 1.14.0)
 - Revocar todos los secretos, API KEYs y credenciales
 - Añadir ignore-scripts=true en el archivo de configuración de NPM (.npmrc)

OWASP TOP 10

A04:2025 Cryptographic Failures



- Aborda cualquier debilidad en el diseño, implementación o gestión de mecanismos criptográficos destinados a garantizar la confidencialidad, integridad o autenticidad de la información.
 - **Impacto Masivo:** Un solo fallo criptográfico puede comprometer la totalidad de los datos sensibles de una aplicación, afectando a miles o millones de usuarios.
 - **Difícil de Detectar:** A menudo, estas vulnerabilidades no son obvias en la interfaz de usuario. Requieren análisis de código estático, dinámico (pruebas automáticas) o revisiones manuales expertas para ser identificadas.
 - **Falsa Sensación de Seguridad:** Usar criptografía "casera" o algoritmos obsoletos da una falsa sensación de protección. Los atacantes especializados pueden romper estos mecanismos rápidamente.

OWASP TOP 10

A04:2025 Cryptographic Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son (1/2):
 - **Transmisión en texto plano:** Enviar datos sensibles a través de protocolos no cifrados (ej. HTTP, FTP, SMTP sin TLS/SSL)
 - **Uso de algoritmos débiles u obsoletos:** Emplear algoritmos que han sido rotos o se consideran inseguros (ej. MD5 o SHA-1 para hashing de contraseñas, DES o RC4 para cifrado, RSA con claves cortas)
 - **Gestión Inadecuada de Claves:** Claves de cifrado "hardcodeadas" en el código fuente, almacenadas en archivos de configuración no protegidos, o sin un proceso de rotación regular.

OWASP TOP 10

A04:2025 Cryptographic Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son (2/2):
 - **Generación de entropía deficiente:** Utilizar funciones de generación de números aleatorios predecibles para crear claves, tokens de sesión o "nonces".
 - **Falta de cifrado en reposo:** Almacenar datos sensibles en bases de datos o sistemas de archivos sin cifrar. Si el servidor es comprometido físicamente o a través de otra vulnerabilidad, los datos son accesibles directamente.
 - **Hashing de contraseñas inseguro:** Almacenar contraseñas usando algoritmos de hashing rápidos (ej. MD5, SHA-256 directo) sin "salt" (sal) o sin factores de trabajo computacionales (como bcrypt, PBKDF2, Argon2), lo que facilita ataques de diccionario.

OWASP TOP 10

A04:2025 Cryptographic Failures



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/2):
 - **Cifrar todo en tránsito:** Implementar HTTPS (TLS/SSL actualizado) para todas las comunicaciones, asegurando configuraciones seguras (cifradores modernos, HSTS).
 - **Cifrar todo en reposo:** Aplicar cifrado fuerte (ej. AES-256) a los datos sensibles almacenados en bases de datos, copias de seguridad y sistemas de archivos.
 - **Usar algoritmos y protocolos estándar:** Utilizar únicamente algoritmos y bibliotecas criptográficas probadas y recomendadas por la comunidad de seguridad (ej. Argon2, bcrypt, AES). Evitar implementar criptografía "casera".

OWASP TOP 10

A04:2025 Cryptographic Failures

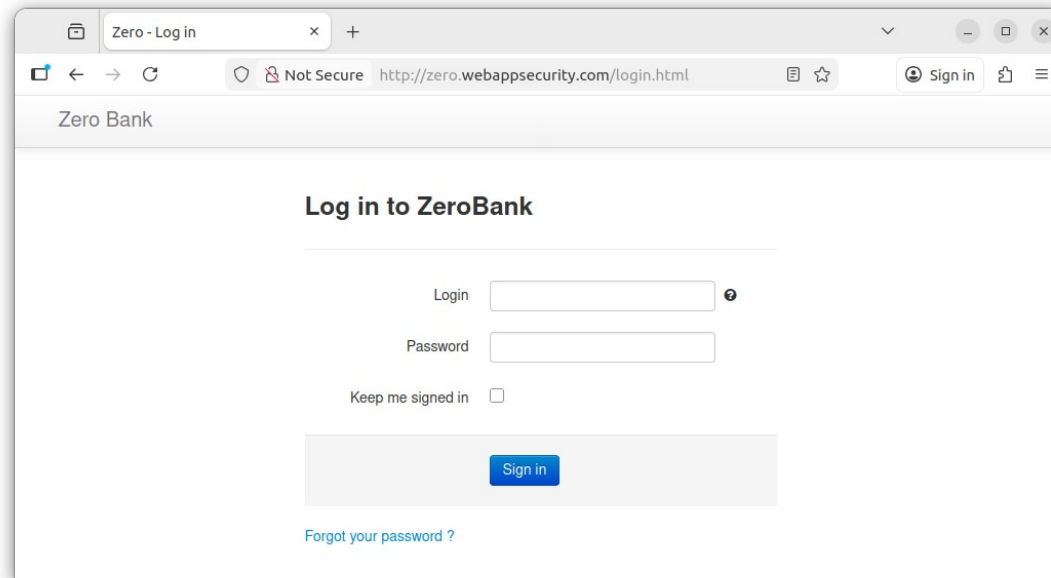


- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/2):
 - **Gestión segura de claves:** Implementar sistemas de gestión de claves (KMS) robustos para almacenar, proteger y rotar claves de cifrado de forma segura. Nunca "hardcodear" claves.
 - **Hashing de contraseñas robusto:** Utilizar algoritmos de hashing específicos para contraseñas, que incluyan un "salt" único por usuario y un factor de trabajo ajustable (ej. Argon2id, bcrypt).
 - **Desactivar protocolos y cifradores inseguros:** Configurar servidores web y de aplicaciones para rechazar conexiones que utilicen versiones antiguas de SSL/TLS o cifradores débiles.

OWASP TOP 10

A04:2025 Cryptographic Failures

- Ejemplo – Página con HTTP



<http://zero.webappsecurity.com/login.html>

*enp3s0

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

http.request.method == "POST"

No.	Time	Source	Destination	Protocol	Length	Info
284	12.620924055	192.168.1.58	54.82.22.214	HTTP	876	POST /signin.html HTTP/1.1 (application/x-www-form-urlencoded)

Frame 284: Packet, 876 bytes on wire (7008 bits), 876 bytes captured (7008 bits)

Ethernet II, Src: MicroStarINT_e0:f1:83 (4c:cc:6a:e0:f1:83), Dst: MitraStarTec_b

Internet Protocol Version 4, Src: 192.168.1.58, Dst: 54.82.22.214

Transmission Control Protocol, Src Port: 58598, Dst Port: 80, Seq: 1, Ack: 1, Len: 876

- Source Port: 58598
- Destination Port: 80
- [Stream index: 27]
- [Stream Packet Number: 4]
- [Conversation completeness: Incomplete, DATA (15)]
- [TCP Segment Len: 810]
- Sequence Number: 1 (relative sequence number)
- Sequence Number (raw): 1922491890
- [Next Sequence Number: 811 (relative sequence number)]
- Acknowledgment Number: 1 (relative ack number)
- Acknowledgment number (raw): 2143679327
- 1000 = Header Length: 32 bytes (8)
- Flags: 0x018 (PSH, ACK)
- Window: 502
- [Calculated window size: 64256]
- [Window size scaling factor: 128]
- Checksum: 0x7114 [unverified]
- [Checksum Status: Unverified]
- Urgent Pointer: 0
- Options: (12 bytes), No-Operation (NOP), No-Operation (NOP), Timestamps
- [Timestamps]
- [SEQ/ACK analysis]
- [Client Contiguous Streams: 1]
- [Server Contiguous Streams: 1]
- TCP payload (810 bytes)
- Hypertext Transfer Protocol
- HTML Form URL Encoded: application/x-www-form-urlencoded
 - Form item: "user_login" = "admin"
 - Form item: "user_password" = "admin"
 - Form item: "submit" = "Sign in"
 - Form item: "user_token" = "8a83d29e-77d5-486b-add6-90156729e45c"

Utilizando herramientas como Wireshark podemos interceptar el tráfico de una red a la que estemos conectados y ver el cuerpo de la petición

0000 2c 96 82 b8 1c 08 4c cc 6a e0 f1 83 08 00 45 00
0010 03 5e cf 4c 40 00 40 06 59 43 c0 a8 01 3a 36 52
0020 16 d6 e4 e6 00 50 72 96 e5 f2 7f c5 f3 5f 80 18
0030 01 f6 71 14 00 00 01 01 08 0a b8 a2 44 bf a3 bf

0180 69 74 2f 35 33 37 2e 33 36 20 28 4b 48 54 4d 4c
0190 2c 20 6c 69 6b 65 20 47 65 63 6b 6f 29 20 43 68
01a0 72 6f 6d 65 2f 31 34 36 2e 30 2e 30 2e 30 20 53
01b0 61 66 61 72 69 2f 35 33 37 2e 33 36 0d 0a 41 63
01c0 63 65 70 74 3a 20 74 65 78 74 2f 68 74 6d 6c 2c
01d0 61 70 70 6c 69 63 61 74 69 6f 6e 2f 78 68 74 6d
01e0 6c 2b 78 6d 6c 2c 61 70 70 6c 69 63 61 74 69 6f
01f0 6e 2f 78 6d 6c 3b 71 3d 30 2e 39 2c 69 6d 61 67
0200 65 2f 61 76 69 66 2c 69 6d 61 67 65 2f 77 65 62
0210 70 2c 69 6d 61 67 65 2f 61 70 6e 67 2c 2a 2f 2a
0220 3b 71 3d 30 2e 38 2c 61 70 70 6c 69 63 61 74 69
0230 6f 6e 2f 73 69 67 6e 65 64 2d 65 78 63 68 61 6e
0240 67 65 3b 76 3d 62 33 3b 71 3d 30 2e 37 0d 0a 52
0250 65 66 65 72 65 72 3a 20 68 74 74 70 3a 2f 2f 7a
0260 65 72 6f 2e 77 65 62 61 70 70 73 65 63 75 72 69
0270 74 79 2e 63 6f 6d 2f 6c 6f 67 69 6e 2e 68 74 6d

.....L. j.....E..
..A..@..YC...:6R
.....Pr.....
.....q.....D...
".POST / signin.h
tml HTTP /1.1..Ho
st: zero .webapps
ecurity. com..Con
nection: keep-al
ive..Con tent-Len
gth: 99..Pragma:
no-cach e..Cache
-Control : no-cac
he..Orig in: http
://zero. webappse
curity.c om..Cont
ent-Type : applic
ation/x- www-form
-urlenco ded..Upg
rade-Ins ecure-Re
quests: 1..User-
Agent: M ozilla/5
.0 (X11; Linux x
86_64) A ppleWebK
it/537.3 6 (KHTML
, like G ecko) Ch
rome/146 .0.0.0 S
afari/53 7.36..Ac
cept: te xt/html,
applicat ion/xhtm
l+xml,ap plicatio
n/xml;q= 0.9,imag
e/avif,i mage/web
p,image/ apng,*/*
;q=0.8,a pplicati
on/signe d-exchan
ge;v=b3; q=0.7..R
eferer: http://z
ero.weba ppsecuri
ty.com/l ogin.htm

wireshark_enp3s044CWN3.pcapng

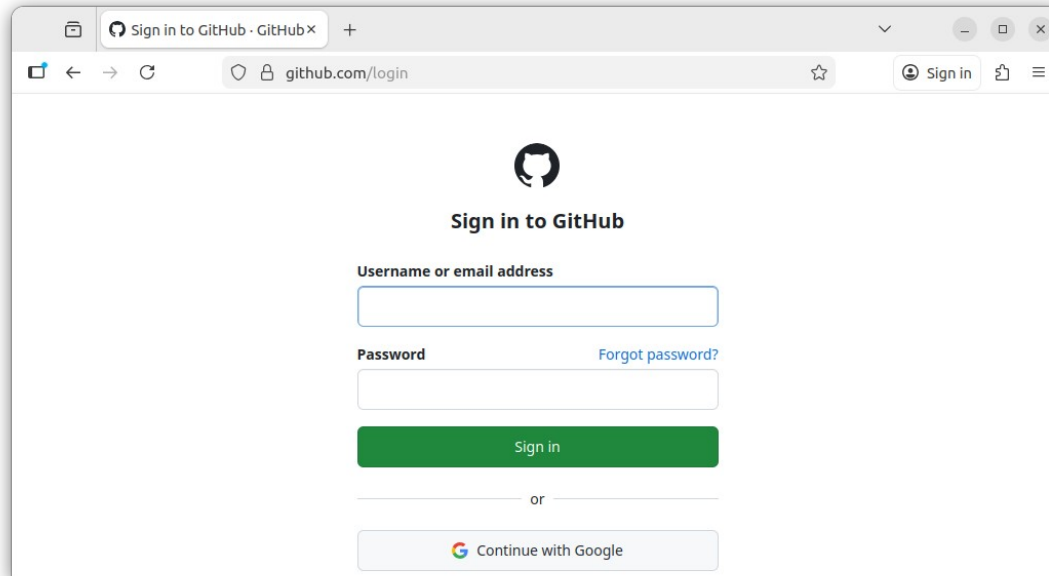
Packets: 429 · Displayed: 1 (0.2%) · Dropped: 0 (0.0%) Profile: Default



OWASP TOP 10

A04:2025 Cryptographic Failures

- Ejemplo – Página con HTTPS



<https://github.com/login>



***enp3s0**

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

tls.app_data and tcp.port == 443

No.	Time	Source	Destination	Protocol	Length	Info
406	9.466370992	192.168.1.58	140.82.113.21	TLSv1.2	193	Application Data
408	9.466434543	192.168.1.58	140.82.113.21	TLSv1.2	126	Application Data
412	9.466651443	192.168.1.58	140.82.113.21	TLSv1.2	1407	Application Data
581	14.504726175	192.168.1.58	140.82.113.21	TLSv1.2	193	Application Data
582	14.504748270	192.168.1.58	140.82.113.21	TLSv1.2	912	Application Data
656	14.826115862	192.168.1.58	140.82.113.21	TLSv1.2	193	Application Data
657	14.826134912	192.168.1.58	140.82.113.21	TLSv1.2	728	Application Data
666	14.913972553	192.168.1.58	140.82.113.21	TLSv1.2	193	Application Data
668	14.913998867	192.168.1.58	140.82.113.21	TLSv1.2	126	Application Data

Frame 412: Packet, 1407 bytes on wire (11256 bits), 1407 bytes captured (11256 bits) on interface enp3s0

Ethernet II, Src: MicroStarINT_e0:f1:83 (4c:cc:6a:e0:f1:83), Dst: MitraStarT...

Internet Protocol Version 4, Src: 192.168.1.58, Dst: 140.82.113.21

Transmission Control Protocol, Src Port: 36122, Dst Port: 443, Seq: 5884, Ack: 3500, Win: 65535, Len: 1407

[4 Reassembled TCP Segments (5613 bytes) ==> 1407 bytes]

Transport Layer

[Stream index: 0]

TLSv1.2 Record

Content Type: Application Data

Version: TLSv1.2

Length: 5608

Encrypted Application Data

[Application Data]

En este caso, podemos interceptar los paquetes dirigidos a github.com (IP 140.82.113.21) pero no podemos obtener ninguna información relevante de los paquetes

Packet (1407 bytes) Reassembled TCP (5613 bytes)

Application Data Protocol (tls.app_data_proto)

Packets: 779 · Displayed: 141 (18.1%) · Dropped: 0 (0.0%) Profile: Default

OWASP TOP 10

A05:2025 Injection



- Aborda varias vulnerabilidades relacionadas con la inyección de código:
 - SQL, NoSQL, comandos, ORM, LDAP ...
- Una vulnerabilidad por inyección es un fallo en una aplicación que permite enviar **datos introducidos por usuarios no fiables a un intérprete** (por ejemplo, un navegador, una base de datos o la línea de comandos) y hace que dicho intérprete **ejecute partes de esos datos como si fueran comandos**.

OWASP TOP 10

A05:2025 Injection



- Los fallos más frecuentes que causan esta vulnerabilidad son:
 - No se validan, filtran o sanitizan los datos proporcionados por el usuario.
 - Las queries dinámicas o las llamadas no parametrizadas se interpretan o ejecutan directamente.
 - Se vulneran los parámetros de búsqueda de los ORM para extraer registros sensibles adicionales.

OWASP TOP 10

A05:2025 Injection



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/2):
 - **Realizar análisis estático de código:** revisar el código fuente manualmente o con herramientas automáticas)
 - **Realizar análisis dinámico de código:** pruebas automáticas de todos los parámetros, cabeceras, URL, cookies, y la entrada de datos vía JSON, SOAP, o XML)
 - **Uso de APIs Seguras:** Evitar en lo posible el uso de intérpretes que requieran concatenar cadenas de texto para formar comandos.

OWASP TOP 10

A05:2025 Injection



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/2):
 - **Consultas parametrizadas (Prepared Statements):** Obliga a la base de datos a tratar la entrada del usuario estrictamente como un dato, independientemente de los caracteres especiales que contenga.
 - **Validación de entrada (Listas Blancas):** Rechazar cualquier dato que no cumpla un formato estricto esperado.
 - **Escape o codificación de datos (Encoding):** Consiste en convertir caracteres especiales en su equivalente inofensivo en HTML (ej. < se convierte en <). Así el navegador lo renderiza como texto, pero no lo ejecuta como código.
 - **Principio de Menor Privilegio en la Base de Datos:** El usuario que usa la aplicación web para conectarse a la base de datos solo debe tener permisos para hacer lo estrictamente necesario (ej. no debería tener permisos para hacer un DROP TABLE).

OWASP TOP 10

A05:2025 Injection – SQL Injection



- Las SQL Injection implican la modificación de las sentencias SQL de las aplicaciones, de forma que un atacante puede modificar una consulta a base de datos a su antojo.
- Pueden llevar a la obtención de datos por parte de usuarios no autenticados o no autorizados.
- Hoy en día casi todos los desarrolladores están concienciados sobre las inyecciones clásicas (' OR '1'='1').

OWASP TOP 10

A05:2025 Injection – SQL Injection



- Inyecciones típicas (1/2):
 - **Basada en errores (Error-based):** El atacante introduce caracteres especiales (como una simple comilla ') buscando que la base de datos "choque" y devuelva un error técnico en la pantalla. Este error suele revelar qué tipo de base de datos se usa (MySQL, PostgreSQL, Oracle) y pistas sobre su estructura.
 - **Basada en UNION (UNION-based):** Es la técnica de robo de datos por excelencia. El operador SQL UNION permite combinar los resultados de dos consultas distintas en una sola tabla. El atacante aprovecha un buscador legítimo (ej. buscar productos) para "pegarle" una segunda consulta que extraiga contraseñas de otra tabla totalmente distinta o la borre

OWASP TOP 10

A05:2025 Injection – SQL Injection



- Inyecciones típicas (2/2):
 - **Booleana (Boolean-based)**: La página responde de forma diferente cuando la consulta a BDD devuelve datos y cuando no. Esto permite extraer datos “por piezas”, con sentencias como LIKE o SUBSTRING
 - **Basada en Tiempo (Time-based)**: Similar a la anterior, pero usando delays de tiempo para diferenciar consultas exitosas de no exitosas.

OWASP TOP 10

A05:2025 Injection – SQL Injection



- Ejemplo – SQLi en Spring
 - Aplicación REST que permite consultar libros (solo operaciones GET)
 - Para la consulta, en lugar del repositorio, usa EntityManager para hacer una consulta a la BBDD

OWASP TOP 10

A05:2025 Injection – SQL Injection



• Ejemplo – SQLi en Spring

security_ejem2_sql

```
@SuppressWarnings("unchecked")
public List<Book> findAll(Integer from, Integer to, String lang) {
    String query = "SELECT * FROM Book";
    if ((from != null && to != null) || isEmptyField(lang)) {
        query += " WHERE";
    }
    if (from != null && to != null) {
        query += " publication_year BETWEEN " + from + " AND " + to;
    }
    if (from != null && to != null && isEmptyField(lang)) {
        query += " AND";
    }
    if (isEmptyField(lang)) {
        query += " lang='" + lang + "'";
    }
    return (List<Book>) entityManager.createNativeQuery(query, Book.class).getResultList();
}
```

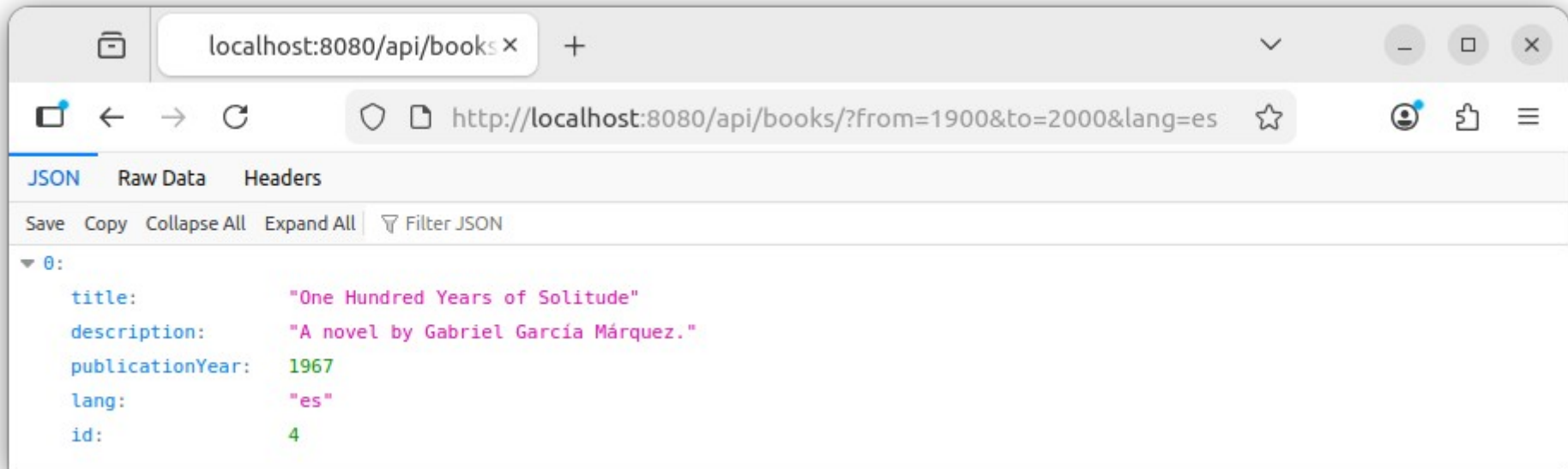
OWASP TOP 10

A05:2025 Injection – SQL Injection



- Ejemplo – SQLi en Spring

security_ejem2_sqli



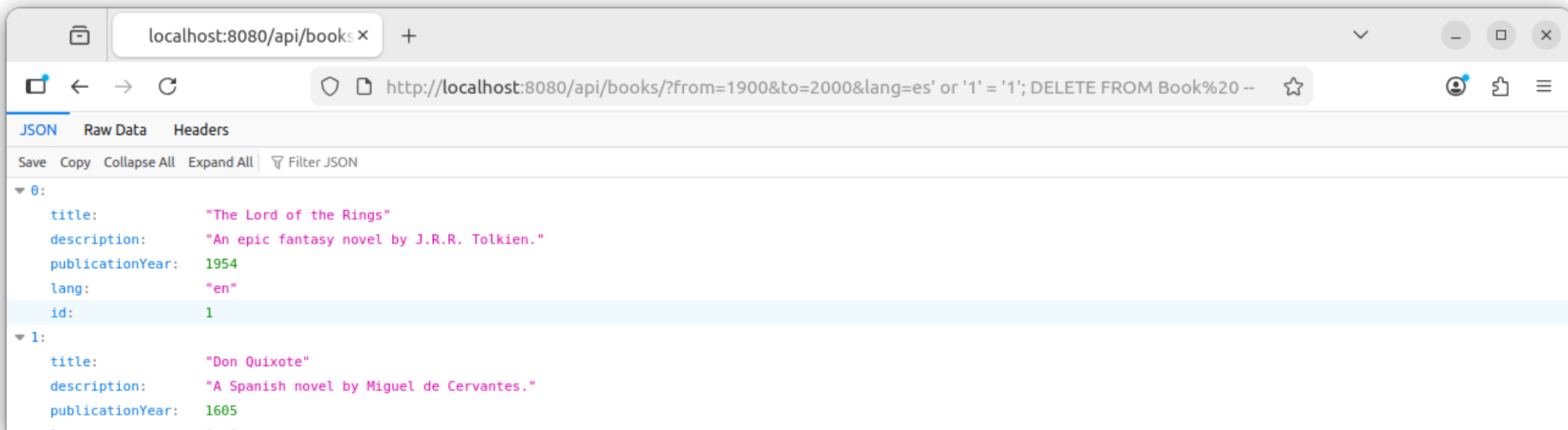
OWASP TOP 10

A05:2025 Injection – SQL Injection



• Ejemplo – SQLi en Spring

security_ejem2_sqli



`http://localhost:8080/api/books/?from=1900&to=2000&lang=es' or '1' = '1'; DELETE FROM Book --`

OWASP TOP 10

A05:2025 Injection – SQL Injection



• Ejemplo – SQLi en Spring

security_ejem2_sql

localhost:8080/api/books × +

http://localhost:8080/api/books/?from=1900&to=2000&lang=es' or '1' = '1'; DELETE FROM Book%20--

JSON Raw Data Headers

Save Copy Collapse All Expand All Filter JSON

```
0:
  title: "The Lord of the Rings"
  description: "An epic fantasy novel by J.R.R. Tolkien."
  publicationYear: 1954
  lang: "en"
  id: 1
1:
  title: "Don Quixote"
  description: "A Spanish novel by Miguel de Cervantes"
```

2026-04-20T20:58:13.345+02:00 DEBUG 152209 --- [nio-8080-exec-1] org.hibernate.SQL : SELECT * FROM Book WHERE publication_year BETWEEN 1900 AND 2000 AND lang='es' or '1' = '1'; DELETE FROM Book --'

http://localhost:8080/api/books/?from=1900&to=2000&lang=es' or '1' = '1'; DELETE FROM Book --

OWASP TOP 10

A05:2025 Injection – SQL Injection



- Ejemplo – SQLi en Spring
 - La solución pasa por usar un ORM (Object-Relational Mapping) en lugar de hacer consultas susceptibles de ser vulnerables
 - El ORM escapa valores peligrosos por defecto
 - Si necesitamos realizar consultas dinámicas, hay otras opciones más seguras ofrecidas por los frameworks

OWASP TOP 10

A05:2025 Injection – XSS



- Los ataques de **Cross-Site Scripting (XSS)** permiten la inyección de código por parte de los usuarios en campos cuyo contenido pueden controlar (cajas de comentarios, texto enriquecido, etc.).
- Es una vulnerabilidad común y peligrosa. Si no protegemos nuestra aplicación adecuadamente, un atacante podría, utilizando JavaScript:
 - Acceder a datos sensibles (cookies de sesión).
 - Enviar peticiones HTTP con contenido arbitrario a destinos arbitrarios.
 - Modificar el DOM de la página actual.

OWASP TOP 10

A05:2025 Injection – XSS



- Existen diferentes tipos de XSS, clasificados en función del fallo que presente la aplicación.
 - XSS Reflejado
 - XSS Persistente
 - XSS basado en DOM

OWASP TOP 10

A05:2025 Injection – XSS Reflejado



- Permite ejecutar código JavaScript malicioso en el navegador de la víctima.
- Se suele encontrar en aplicaciones que incluyen campos cuyo valor se imprime por pantalla (“Hola, <username>”).
- Para perpetrarlo, el atacante debe crear una URL maliciosa y hacer que la víctima acceda a ella.

OWASP TOP 10

A05:2025 Injection – XSS Reflejado



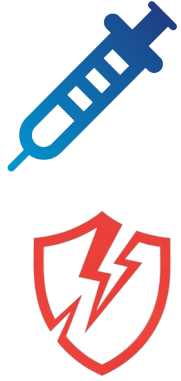
- Ejemplo – Aplicación Spring

`security_ejem3_xss_reflected`

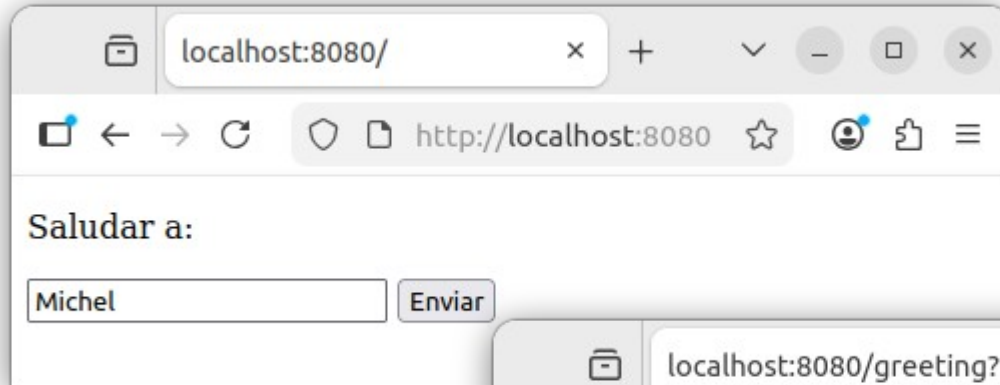
- Aplicación muy simple que recibe una cadena de texto y la muestra en otra página
- Se utiliza Mustache como motor de plantillas y se renderiza el mensaje utilizando {{{ }}}, lo que permite insertar HTML de manera dinámica

OWASP TOP 10

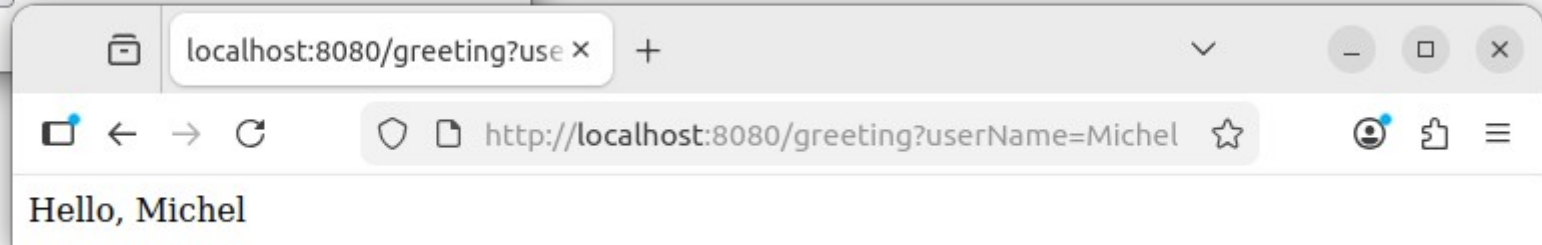
A05:2025 Injection – XSS Reflejado



- Ejemplo – Aplicación Spring



security_ejem3_xss_reflected

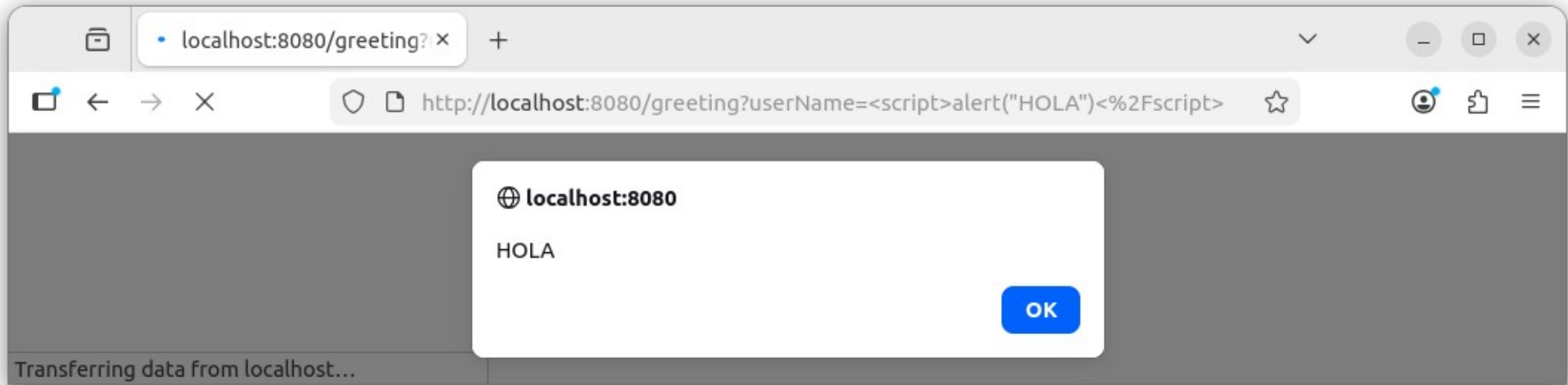


OWASP TOP 10

A05:2025 Injection – XSS Reflejado



- Ejemplo – Aplicación Spring security_ejem3_xss_reflected



`http://localhost:8080/greeting?userName=<script>alert("HOLA")</script>`

OWASP TOP 10

A05:2025 Injection – XSS Reflejado



- Ejemplo – Aplicación Spring `security_ejem3_xss_reflected`

- La solución pasa por no utilizar {{{ }}}
- Es simple, pero algunos motores de plantillas antiguos pueden estar mal configurados y permitir el renderizado de HTML potencialmente malicioso

OWASP TOP 10

A05:2025 Injection – XSS Persistente



- Permite ejecutar código JavaScript malicioso en el navegador de cualquier usuario que acceda al recurso que contiene almacenado dicho código (comentarios, posts, etc.).
- El atacante no necesita personalizar el ataque a una víctima, sino que todos los visitantes de un determinado endpoint serán víctimas sin necesidad de tomar acción.

OWASP TOP 10

A05:2025 Injection – XSS Persistente



- Ejemplo – Aplicación Spring

security_ejem4_xss_persisted

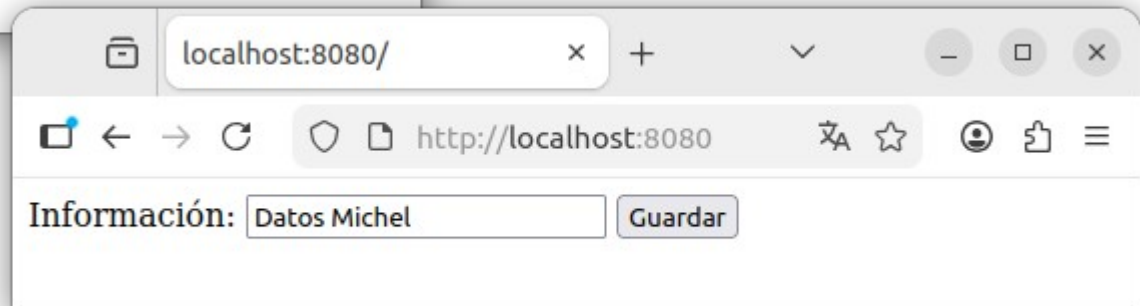
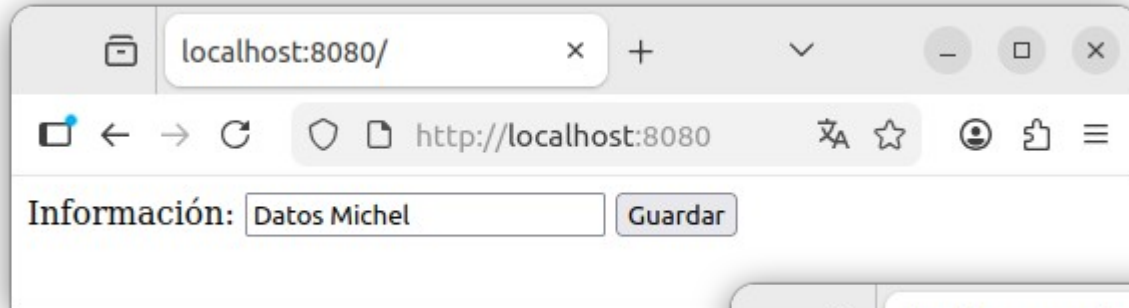
- Aplicación muy simple que recibe una cadena de texto, la guarda y la muestra en otra página
- Se utiliza Mustache como motor de plantillas y se renderiza el mensaje utilizando {{{ }}}, lo que permite insertar HTML de manera dinámica

OWASP TOP 10

A05:2025 Injection – XSS Persistente



- Ejemplo – Aplicación Spring `security_ejem4_xss_persisted`

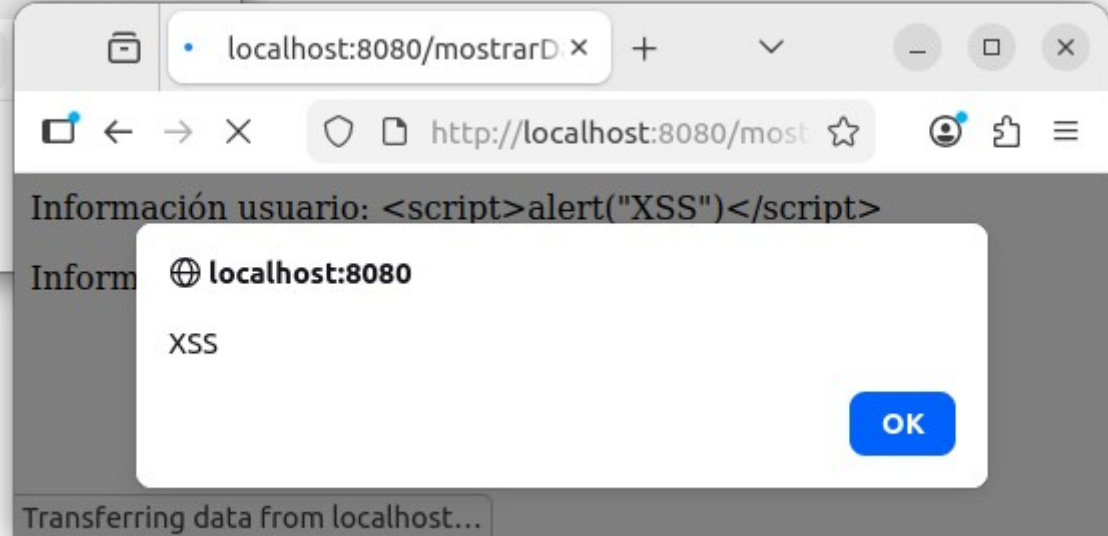
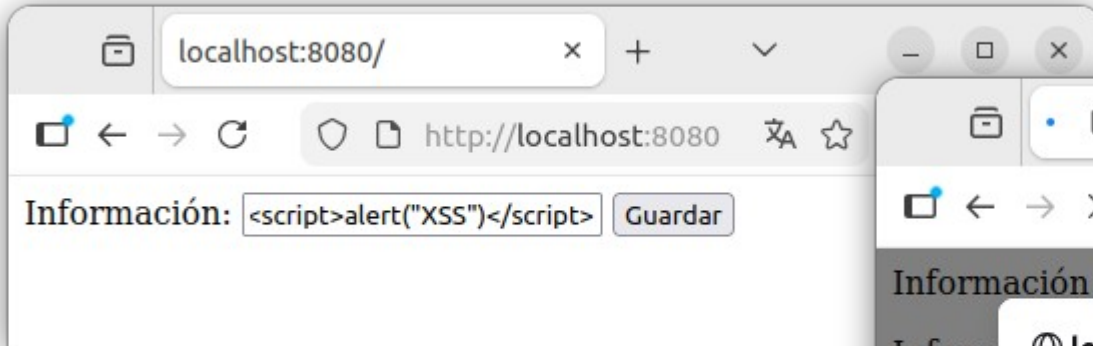


OWASP TOP 10

A05:2025 Injection – XSS Persistente



- Ejemplo – Aplicación Spring `security_ejem4_xss_persisted`



OWASP TOP 10

A05:2025 Injection – XSS Persistente



- Ejemplo – Aplicación Spring `security_ejem4_xss_persisted`
 - La solución pasa (de nuevo) por no utilizar
`{{{ }}}`

OWASP TOP 10

A05:2025 Injection – XSS Persistente



- Es posible que sea inevitable tener que inyectar código HTML (p.e. texto enriquecido)
- Debemos sanitizar la entrada del usuario con librerías
 - OWASP Java HTML Sanitizer
 - <https://github.com/OWASP/java-html-sanitizer>
 - Permite definir políticas de sanitización para filtrar elementos indeseados
 - Es conveniente utilizar “white lists” (elementos/configuraciones que permitimos) sobre “black lists” (elementos/configuraciones que no permitimos)
 - No todas las inyecciones son con <script>

OWASP TOP 10

A05:2025 Injection – XSS basado en DOM



- El DOM es una API que permite transformar el código HTML de una web en una estructura de árbol jerárquico.
- Si no se controla el input del usuario, podría llegar a ejecutar código JavaScript en el navegador a través del DOM.
- Este ataque es similar al XSS reflejado, con la diferencia de que la vulnerabilidad se encuentra en el propio DOM de la web y no en una de las entradas de datos de esta.
 - El servidor no está involucrado

OWASP TOP 10

A05:2025 Injection – XSS basado en DOM



- Ejemplo – HTML que utiliza el DOM con JS

```
<h1>Panel de Usuario</h1>
```

security_ejem5_xss_dom

```
<div id="mensaje-bienvenida">Cargando...</div>
```

```
<script>
```

```
  let nombreUsuario = decodeURIComponent(window.location.hash.substring(1));
```

```
  if (!nombreUsuario) {  
    nombreUsuario = "Invitado";  
  }
```

```
  let divSaludo = document.getElementById("mensaje-bienvenida");  
  divSaludo.innerHTML = "Hola, " + nombreUsuario + ". ¡Qué bueno verte!";
```

```
</script>
```

OWASP TOP 10

A05:2025 Injection – XSS basado en DOM



- Ejemplo – HTML que utiliza el DOM con JS



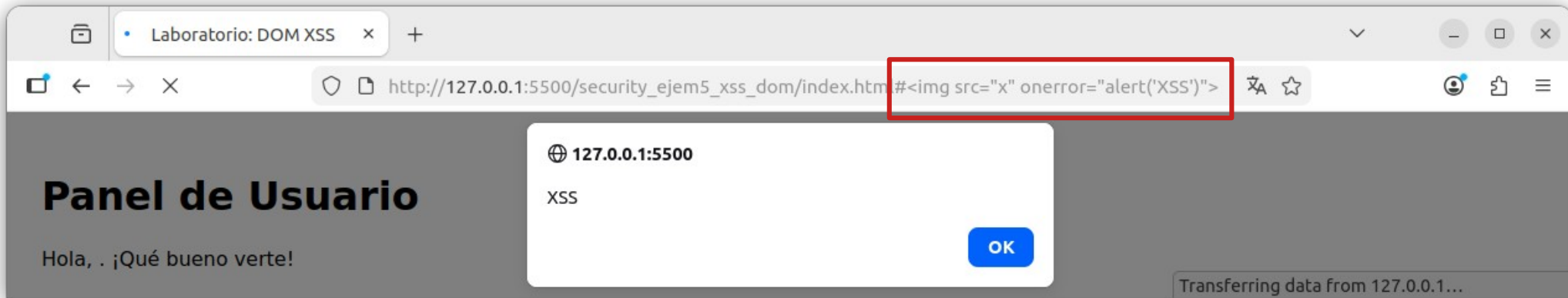
OWASP TOP 10

A05:2025 Injection – XSS basado en DOM



- Ejemplo – HTML que utiliza el DOM con JS

security_ejem5_xss_dom



`http://127.0.0.1:5500/security_ejem5_xss_dom/index.html#<img src='x' onerror='alert('XSS')'>`

OWASP TOP 10

A05:2025 Injection – XSRF



- Los ataques de **Cross-Site Request Forgery (XSRF o CSRF)** permiten impersonar a un usuario explotando la confianza que deposita un sitio en particular en un usuario autenticado (habitualmente, utilizando un token de sesión o una cookie).
- Un ataque de XSS puede aprovecharse como trampolín para un XSRF.

OWASP TOP 10

A05:2025 Injection – XSRF



- Un XSRF funciona de la siguiente manera:
 - Una víctima hace login en un servicio de manera legítima.
 - La cookie queda almacenada en el navegador de la víctima cuando inicia sesión.
 - De algún modo (XSS, phishing, ingeniería social, etc.), la víctima es dirigida a un sitio malicioso controlado por el atacante.
 - El sitio malicioso envía una petición al sitio legítimo en el que el usuario estaba logado previamente, produciéndose la impersonación.

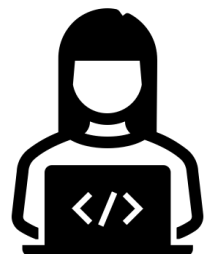
OWASP TOP 10

A05:2025 Injection – XSRF

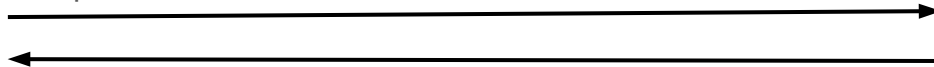


<https://entradasdeconciertos.com/cambiar-password>

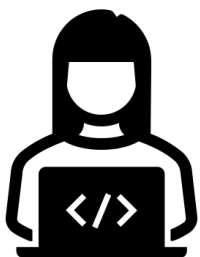
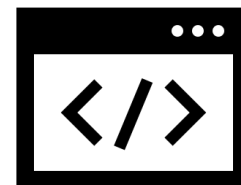
Cookie: ab34fe7809cc1233deadbeef



<https://entradasdeconciertos.com/>

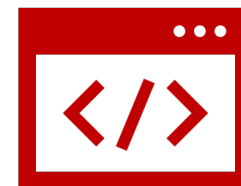


Cookie: ab34fe7809cc1233deadbeef



<https://entradasdeconcierts.com/entrada-gratis>

Cookie: ab34fe7809cc1233deadbeef



OWASP TOP 10

A05:2025 Injection – XSRF



- Antes de entrar en detalle sobre las vías de mitigación de CSRF (o XSRF), es importante destacar que **todas las defensas contra esta vulnerabilidad se pueden saltar si tenemos una vulnerabilidad de XSS.**
- Si un atacante puede exfiltrar la cookie del usuario, por ejemplo, puede llevar a cabo todas las acciones que podría llevar a cabo con XSRF.
- Vídeo de ejemplo:
<https://www.youtube.com/watch?v=DWThhRPwGEg>

OWASP TOP 10

A05:2025 Injection – XSRF



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en:
 - **Aplicar las protecciones por defecto del framework y utilizarlas.**
 - Si el framework no dispone de esta característica, **añadir tokens CSRF a todas las peticiones que cambian el estado** (provocan acciones en la aplicación) y validarlas en el backend.
 - No usar solicitudes GET para operaciones que modifican el estado.

OWASP TOP 10

A05:2025 Injection – Path Traversal



- La vulnerabilidad Path Traversal permite al atacante navegar a **rutas del sistema de ficheros** para leer o escribir en ficheros a los que de normal no tendría acceso
- Podría considerarse también en la categoría **A01:2025 Broken Access Control**
- Puede dar lugar a dos vulnerabilidades concretas: **Local File Inclusion (LFI)** o **Arbitrary Write**

OWASP TOP 10

A05:2025 Injection – Path Traversal



- La vulnerabilidad **Local File Inclusion (LFI)** permite al atacante cargar un archivo local para leerlo o ejecutarlo
 - Acceso o ejecución de recursos locales
- El objetivo suele ser incluir ficheros de credenciales (como /etc/passwd) para poder leerlos
- Es común que se combine con Path Traversal, aunque no es un requisito

OWASP TOP 10

A05:2025 Injection – Path Traversal



- La vulnerabilidad **Arbitrary Write** consiste en que el atacante escribe o modifica un fichero en una ruta en la que no debería tener permisos
- Aprovecha las rutas conocidas con un Path Traversal
- Suele combinarse con LFI, que podría forzar su ejecución

OWASP TOP 10

A05:2025 Injection – Path Traversal



- Ejemplo – Aplicación Spring
 - App sencilla que permite escribir y leer ficheros en disco
 - No controla las rutas dónde se leen y guardan los ficheros

`security_ejem6_path_traversal`

OWASP TOP 10

A05:2025 Injection – Path Traversal



security_ejem6_path_traversal

OWASP TOP 10

A05:2025 Injection – Path Traversal



```
maes@maes-GE72-7RE: ~  
maes@maes-GE72-7RE:~$ curl http://localhost:8080/download_image?imageName=../pom.xml  
<?xml version="1.0" encoding="UTF-8"?>  
<project xmlns="http://maven.apache.org/POM/4.0.0"  
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"  
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/  
xsd/maven-4.0.0.xsd">  
  <modelVersion>4.0.0</modelVersion>  
  
  <groupId>es.urjc.code</groupId>  
  <artifactId>security_ejem6_path_traversal</artifactId>  
  <version>0.1.0</version>  
</project>
```

Hemos conseguido leer un
fichero al que, a priori, no
deberíamos tener acceso

security_ejem6_path_traversal

Burp Suite Community Edition v2023.3.2 - Temporary Project

Dashboard Target **Proxy** Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

Intercept HTTP history WebSockets history Match and replace Proxy settings

No proxy listeners are currently running **Configure**

Intercept on **Forward** Drop Request to http://localhost:8080 [127.0.0.1] Open browser

Time	Type	Direction	Method	URL	Status code	Length
16:40:48 2...	HTTP	→ Request	POST	http://localhost:8080/upload_image		

Request

Pretty Raw Hex

```
1 POST /upload_image HTTP/1.1
2 Host: localhost:8080
3 Content-Length: 23863
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost:8080
10 Content-Type: multipart/form-data; boundary=---WebKitFormBoundaryCBWja57RH8FBBRi4
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/120.0.0.0 Safari/537.36
13 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/png;v=b3;q=0.7
14 Sec-Fetch-Site: same-origin
15 Sec-Fetch-Mode: navigate
16 Sec-Fetch-User: ?1
17 Sec-Fetch-Dest: document
18 Referer: http://localhost:8080/upload_image.html
19 Accept-Encoding: gzip, deflate, br
20 Connection: keep-alive
21
22 -----WebKitFormBoundaryCBWja57RH8FBBRi4
23 Content-Disposition: form-data; name="image"; filename="codeurjc_logo.jpg"
24 Content-Type: image/jpeg
```

localhost:8080/upload_image.html

Image file:

Choose File codeurjc_logo.jpg Save

Utilizando BURP Suite podemos modificar el nombre del fichero que envíamos interceptando la petición

security_ejem6_path_traversal

Event log (1) All issues

Memory: 151.6MB of 7.77GB Disabled



Burp Suite Community Edition v2026.3.2 - Temporary Project

Dashboard Target Proxy Intruder Repeater Collaborator Sequencer Decoder Comparer Logger Organizer Extensions Discover

Intercept HTTP history WebSockets history Match and replace Proxy settings

No proxy listeners are currently running Configure

Intercept on Forward Drop

Request to http://localhost:8080 [127.0.0.1] Open browser

Time	Type	Direction	Method	URL	Status code	Length
16:40:48 2...	HTTP	→ Request	POST	http://localhost:8080/upload_image		

Request

Pretty Raw Hex

```
1 POST /upload_image HTTP/1.1
2 Host: localhost:8080
3 Content-Length: 23863
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: ?0
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: en-US,en;q=0.9
9 Origin: http://localhost:8080
10 Content-Type: multipart/form-data; boundary=---WebKitFormBoundaryCBWja
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML,
13 Accept:
14 text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/
15 hange;v=b3;q=0.7
16 Sec-Fetch-Site: same-origin
17 Sec-Fetch-Mode: navigate
18 Sec-Fetch-User: ?1
19 Referer: http://localhost:8080/upload_image.html
20 Accept-Encoding: gzip, deflate, br
21 Connection: keep-alive
22 -----WebKitFormBoundaryCBWja57RH8FB8Ri4
23 Content-Disposition: form-data; name="image"; filename="../codeurjc_logo.jpg"
24 Content-Type: image/jpeg
```

localhost:8080/upload_image.html

Image file:

Choose File codeurjc_logo.jpg Save

Request headers 19

Utilizando BURP Suite podemos modificar el nombre del fichero que envíamos interceptando la petición

security_ejem6_path_traversal

Event log (1) All issues

Memory: 151.6MB of 7.77GB Disabled



OWASP TOP 10

A05:2025 Injection – Path Traversal



```
✓ security_ejem6_path_traversal
  > images
  > src/main
  > target
  .gitignore
  codeurjc_logo.jpg
  pom.xml
```

Hemos conseguido escribir un fichero en un directorio en el que, a priori, no deberíamos tener acceso

security_ejem6_path_traversal

OWASP TOP 10

A05:2025 Injection – Path Traversal



- Para evitar este tipo de inyecciones podemos sanitizar la entrada del usuario
 - Eliminamos dobles puntos (..) y barras (\\) dejando solo el nombre del archivo
- Es preferible realizar un mejor diseño de la aplicación
 - **Al guardar** - Generar un nuevo nombre en el fichero (usando, por ejemplo, un identificador de bbdd) si el nombre del archivo original no es relevante
 - MAL: nombre original.jpg → Creado por el usuario
 - MEJOR: imagen_1.jpg → Generado por la aplicación
 - **Al leer** - Al devolver un fichero al usuario, utilizar un identificado de la base de datos para recuperarlo en lugar del nombre
 - MAL: /image/?imageName=imagen.jpg → El usuario establece la ruta
 - MEJOR: /image/1 → El usuario solo puede indicar el identificado

OWASP TOP 10

A05:2025 Injection



- Como conclusiones generales sobre las inyecciones:
 - **SIEMPRE** debemos validar y sanitizar la entrada del usuario y actuar ante elementos extraños (una petición mal formada, caracteres especiales, entradas inesperadas, etc.)
 - La validación se debe dar **SIEMPRE** en el lado del servidor, **NUNCA** en el lado del cliente.
 - Validar en el lado del cliente es equivalente a confiar a ciegas en los datos que introduce el usuario.
 - Estas validaciones deben aplicarse tanto en core de la aplicación, como en el resto de los servicios (API REST, accesos a bases de datos, etc.)



OWASP TOP 10

A06:2025 Insecure Design

- Una aplicación con un "Diseño Inseguro" puede tener un código escrito a la perfección, sin un solo bug técnico, y aun así ser completamente vulnerable porque la lógica de negocio o la arquitectura son defectuosas.
- En el ciclo de desarrollo de software, la seguridad debe integrarse en las fases más tempranas (diseño y arquitectura), no solo al final durante las pruebas.
 - Si no modelas las amenazas antes de escribir el código, terminarás con un **diseño inseguro**.

OWASP TOP 10

A06:2025 Insecure Design



- Los fallos más frecuentes que causan esta vulnerabilidad son (1/2):
 - **Confianza en el cliente (Client-Side Trust):** Diseñar un e-commerce donde el navegador del usuario calcula el precio final y se lo envía al servidor para cobrar.
 - El código funciona bien, pero el diseño permite a un atacante interceptar la petición y decir "este televisor cuesta 1€".
 - **Flujos de recuperación débiles:** Diseñar un sistema de "He olvidado mi contraseña" basado en "Preguntas de Seguridad" (ej. ¿Cuál es el nombre de tu primera mascota?).
 - Esto es un diseño inseguro hoy en día, ya que esa información es pública en redes sociales.

OWASP TOP 10

A06:2025 Insecure Design



- Los fallos más frecuentes que causan esta vulnerabilidad son (2/2):
 - **Falta de límites de negocio (Rate Limiting):** Un sistema de reservas de cine que te permite añadir 500 butacas a tu carrito y retenerlas durante 15 minutos sin pagar.
 - Un atacante puede crear un bot para "reservar" todo el cine constantemente, haciendo una Denegación de Servicio (DoS) a nivel de negocio.
 - **Carencia de modelado de amenazas:** Crear un foro donde los usuarios pueden subir archivos, pero no pensar desde la fase de diseño que alguien intentará subir un archivo ejecutable malicioso (.exe o .php) disfrazado de .jpg.
 - Relacionado con [A05:2025 - Injection](#)

OWASP TOP 10

A06:2025 Insecure Design



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/2):
 - **Modelado de amenazas (Threat Modeling):** Reunirse antes de programar y preguntarse sistemáticamente:
 - ¿Cómo podría alguien abusar de esta funcionalidad?
 - Uso de metodologías como STRIDE
 - <https://www.purestorage.com/la/knowledge/stride-threat-model.html>
 - **Validación en el servidor (El servidor es la autoridad):** Diseñar siempre bajo la premisa de que el cliente (navegador/app) está en manos del enemigo. El servidor siempre debe recalcular precios, verificar permisos y validar estados.

OWASP TOP 10

A06:2025 Insecure Design



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/2):
 - **Límites (Rate Limiting):** Diseñar las APIs de negocio con límites estrictos.
 - ¿Cuántas veces por minuto puede un usuario intentar adivinar un código de descuento? → Máximo 5
 - **Uso de patrones de diseño seguros:** En lugar de inventar flujos críticos desde cero (como la autenticación o la recuperación de contraseñas), utilizar patrones estandarizados y probados por la industria.



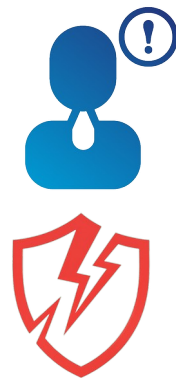
OWASP TOP 10

A07:2025 Authentication Failures

- Confirmar la identidad del usuario y el manejo de la sesión son **elementos críticos** para protegernos ante ataques relacionados con la autenticación.
 - **Acceso directo y total:** Si un atacante roba una sesión o adivina una contraseña, no necesita buscar vulnerabilidades complejas de inyección.
 - **Automatización del ataque:** Los atacantes no prueban contraseñas a mano. Usan listas masivas de credenciales filtradas de otras webs (Credential Stuffing) y bots que prueban miles de contraseñas por segundo (Fuerza Bruta).
 - **Falsa sensación de seguridad:** Muchos desarrolladores creen que con pedir usuario y contraseña es suficiente, olvidando que la sesión posterior (la cookie) es igual de valiosa que la contraseña misma.

OWASP TOP 10

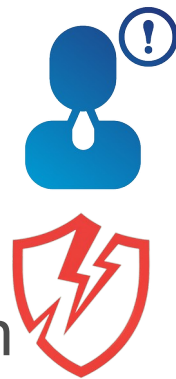
A07:2025 Authentication Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son (1/2):
 - **Ataques de fuerza bruta y credential stuffing:** Permitir que una misma IP o usuario intente iniciar sesión miles de veces seguidas sin bloquearle o pedirle un CAPTCHA.
 - **Políticas de contraseñas débiles:** Permitir a los usuarios registrarse con contraseñas inseguras como 123456, password o qwerty.
 - **Ausencia de Multi-Factor Authentication (MFA):** Depender únicamente de una contraseña en sistemas que manejan datos sensibles.

OWASP TOP 10

A07:2025 Authentication Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son (2/2):
 - **Exposición del token de sesión en la URL:** Enviar el token de sesión en la barra de direcciones (ej. `app.com/dashboard?session_id=123`). Si la víctima copia el enlace y se lo envía a un compañero, le está enviando su sesión abierta.
 - **Sesiones Infinitas:** No implementar un tiempo de expiración o no destruir la sesión en el servidor cuando el usuario pulsa "Cerrar sesión".

OWASP TOP 10

A07:2025 Authentication Failures



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/2):
 - **Implementar MFA (Autenticación Multifactor):** Es la medida más efectiva. Exigir un segundo factor (SMS, App Authenticator, Llave FIDO) hace que las contraseñas robadas sean inútiles por sí solas.
 - **Límites de intentos (Rate Limiting) y bloqueos:** Bloquear temporalmente la cuenta o la IP después de un número determinado de intentos fallidos de inicio de sesión.

OWASP TOP 10

A07:2025 Authentication Failures



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/2):
 - **Comprobación de contraseñas comprometidas:** Utilizar bases de datos (como Have I Been Pwned) durante el registro para prohibir contraseñas que ya han sido expuestas en filtraciones conocidas.
 - **Gestión segura de sesiones:** Generar siempre un ID de sesión nuevo y aleatorio tras un login exitoso.
 - Invalidar las sesiones antiguas.
 - Usar cookies seguras (flags Secure y HttpOnly).
 - Establecer tiempos de inactividad lógicos.



OWASP TOP 10

A07:2025 Authentication Failures

- Ejemplo – URL con token expuesto

`https://vulnerablehost.com/authuser?user=bob&authz\_token=1234&expire=1500000000`

- Los parámetros *user*, *authz_token* y *expire* quedan expuestos con HTTPS en/si:
 - Referer Header (campo que indica la URL de la web anterior)
 - Logs del servidor web
 - Historial y caché del navegador
 - Ante una mirada indiscreta
 - Si comparto (por error) el enlace

OWASP TOP 10



A08:2025 Software or Data Integrity Failures

- Esta categoría agrupa cualquier situación en la que la infraestructura o la aplicación no protegen contra violaciones de integridad.
 - **Infecciones silenciosas:** Si tu aplicación descarga una actualización automáticamente y no verifica si fue modificada por un atacante en el camino, instalará malware con los máximos privilegios del sistema.
 - **Ejecución Remota de Código (RCE):** En el caso de la deserialización insegura, un atacante puede enviar un objeto de datos manipulado que, al ser "reconstruido" por el servidor, ejecuta comandos maliciosos automáticamente.
 - **Compromiso a gran escala:** Si un atacante logra modificar un script alojado en un CDN que usan miles de páginas web, comprometerá a todos los visitantes de esas miles de páginas simultáneamente → Relacionado con **A03:2025 Software Supply Chain Failures**

OWASP TOP 10

A08:2025 Software or Data Integrity Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son (1/2):
 - **Actualizaciones de software sin firmar:** Una aplicación de escritorio descarga su nueva versión por HTTP, o incluso por HTTPS, pero la instala inmediatamente sin comprobar una firma criptográfica que demuestre que proviene del desarrollador original.
 - **Falta de integridad de subrecursos (SRI):** Tu página web carga una librería JS (React o jQuery) desde un CDN externo (ej. `<script src="https://cdn..."></script>`). Si un hacker vulnera ese CDN e inyecta código malicioso en jQuery, tu web lo ejecutará.

OWASP TOP 10

A08:2025 Software or Data Integrity Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son (2/2):
 - **Deserialización insegura:** La aplicación guarda el perfil del usuario convirtiendo el "Objeto" de programación en una cadena de texto (serialización) y lo mete en una cookie. El usuario modifica ese texto para añadir permiso=ejecutar_comandos. Al volver al servidor, la aplicación reconstruye el objeto (deserialización) con el permiso cambiado
 - **Pipelines CI/CD inseguros:** El proceso que compila el código y lo sube a producción no verifica los hashes (huellas digitales) de los artefactos que está moviendo.

OWASP TOP 10

A08:2025 Software or Data Integrity Failures



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/2):
 - **Firmas digitales obligatorias:** Exigir que todas las actualizaciones de software, instaladores y parches estén firmados criptográficamente. Si la firma no coincide, la instalación se aborta.
 - **Integridad de subrecursos (SRI) en el HTML:** Añadir el atributo integrity a las etiquetas <script> que cargan recursos externos. Este atributo contiene el hash exacto del archivo legítimo. Si el CDN envía un archivo alterado, el navegador se negará a ejecutarlo.

OWASP TOP 10

A08:2025 Software or Data Integrity Failures



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/2):
 - **No deserializar datos no confiables:** Utilizar formatos de datos puros y seguros como JSON en lugar de serializar objetos nativos (como hace Java o PHP con `unserialize()`). Si hay que serializar objetos, deben ir firmados digitalmente para evitar su manipulación.
 - **Repositorios confiables:** Asegurar que el pipeline de construcción de software (CI/CD) verifica la integridad de cada contenedor y librería descargada antes de usarla.

OWASP TOP 10

A08:2025 Software or Data Integrity Failures



- Ejemplo – CDN Bootstrap

```
<link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/css/bootstrap.min.css" rel="stylesheet"
integrity="sha384-sRIl4kxILFvY47J16cr9ZwB07vP4J8+LH7qKQnuqkuIAvNWLzeN8tE5YBujZqJLB"
crossorigin="anonymous">
<script src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.8/dist/js/bootstrap.bundle.min.js"
integrity="sha384-FKyoEForCGlyvw9Hj09JcYn3nv7wiPVLz7YYwJrWVcXK/BmnVDxM+D2scQbITxI"
crossorigin="anonymous"></script>
```

El script de Bootstrap (distribuido a través de un CDN) lleva una firma que el browser usa para comprobar que no se ha modificado el código recibido y que se cargará

https://www.w3schools.com/tags/att_script_integrity.asp

<https://getbootstrap.com/docs/5.3/getting-started/download/>



OWASP TOP 10

A09:2025 Security Logging & Alerting Failures

- Una aplicación segura debe registrar (log) todo evento relevante de seguridad de forma inalterable y, lo más importante, debe tener un sistema que alerte cuando detecte un patrón anómalo.
 - **Imposibilidad de respuesta a incidentes:** Cuando ocurra una brecha, los equipos de ciberseguridad necesitan hacer análisis forense. Si no hay logs, es imposible saber qué datos se robaron, cómo entraron los atacantes o si siguen dentro.
 - **Persistencia del atacante:** La mayoría de las brechas no son detectadas por la propia empresa, sino por terceros. Los atacantes confían en la falta de monitorización para extraer datos poco a poco.
 - **Cumplimiento legal:** Leyes como el RGPD (GDPR) o normativas como PCI-DSS exigen estrictamente mantener registros de acceso a datos sensibles. No tenerlos resulta en multas millonarias.

OWASP TOP 10

A09:2025 Security Logging & Alerting Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son(1/2):
 - **Eventos críticos no registrados:** No guardar un registro cuando alguien inicia sesión (éxito o fallo), cambia su contraseña, eleva sus privilegios o realiza una transacción de alto valor.
 - **Logs locales vulnerables:** Guardar los archivos de registro en el mismo servidor donde se ejecuta la aplicación. Lo primero que hace un hacker al comprometer un servidor es ejecutar `rm -rf /var/log/*` para borrar sus acciones.

OWASP TOP 10

A09:2025 Security Logging & Alerting Failures



- Los fallos más frecuentes que causan esta vulnerabilidad son(2/2):
 - **Falta de alertas (Fatiga de Alertas):** El servidor genera miles de líneas de texto por minuto, pero nadie las analiza. Si hay 50,000 intentos de login fallidos desde una IP en Rusia o Corea del Norte a las 3:00 AM y a nadie le suena el teléfono, el sistema ha fallado.
 - **Registro de datos sensibles (Log Injection/Leakage):** Imprimir contraseñas en texto plano, números de tarjeta de crédito o tokens de sesión dentro de los archivos de log.

OWASP TOP 10

A09:2025 Security Logging & Alerting Failures



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/2):
 - **Registro centralizado (SIEM):** Enviar los logs en tiempo real a un servidor externo, seguro y dedicado (como Splunk, ELK Stack, o Datadog). Si el atacante compromete el servidor web, no podrá borrar los logs que ya se enviaron al servidor central.
 - **Logs contextuales e íntegros:** Asegurarse de que cada registro incluya "Quién, Qué, Cuándo y Desde Dónde" (IP, Timestamp, User ID, Acción). Usar formatos estructurados como JSON para que las máquinas puedan leerlos y procesarlos fácilmente.

OWASP TOP 10

A09:2025 Security Logging & Alerting Failures



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/2):
 - **Alertas proactivas basadas en umbrales:** Configurar el sistema centralizado para que envíe un email, un mensaje de Slack o un SMS a los administradores si detecta anomalías (ej. "Más de 10 logins fallidos a cuentas de administrador en 1 minuto").
 - **Sanitización de logs:** Asegurarse de enmascarar o eliminar cualquier dato personal identificable o credencial antes de escribir la línea en el log.



OWASP TOP 10

A10:2025 Mishandling of Exceptional Conditions

- Se produce cuando los programas no logran prevenir, detectar y responder a situaciones inusuales e impredecibles, lo que da lugar a fallos del sistema, comportamientos inesperados y, en ocasiones, vulnerabilidades.
- El principio fundamental de esta vulnerabilidad se resume en una pregunta: **¿Qué hace tu aplicación cuando ocurre algo que no estaba previsto?** (Por ejemplo, se cae la base de datos, un archivo no existe, o el usuario introduce letras donde debían ir números).
- Si la aplicación "entra en pánico" y devuelve todo su estado interno en la pantalla del usuario, o si ante la duda decide permitir el acceso, estamos ante un manejo incorrecto de excepciones.

OWASP TOP 10

A10:2025 Mishandling of Exceptional Conditions



- Los fallos más frecuentes que causan esta vulnerabilidad son (1/2):
 - **Stack Traces visibles en producción:** Dejar el "Modo Depuración" activado en un servidor público. Si un usuario fuerza un error, verá líneas de Excepción → Relacionado con **A02:2025 Security Misconfiguration**
 - **Mensajes de Error que Revelan Estado:** En un formulario de login, devolver el mensaje "La contraseña es incorrecta" (revelando que el usuario sí existe) en lugar de "Usuario o contraseña incorrectos".

OWASP TOP 10

A10:2025 Mishandling of Exceptional Conditions



- Los fallos más frecuentes que causan esta vulnerabilidad son (2/2):
 - **Fallo Abierto (Fail Open):** Un bloque de código como:
 - `try { verificar_permisos(); } catch (Exception e) {
// No hacer nada, continuar }`
 - Si hay un error, el programa continúa como si el usuario tuviera permisos.
 - **Caídas de la aplicación (Crash):** Un atacante envía un archivo PDF corrupto a sabiendas de que la librería de lectura de PDF del servidor no tiene manejo de excepciones. El servidor intenta leerlo, falla y el proceso se cierra por completo, dejando al resto de usuarios sin servicio

OWASP TOP 10

A10:2025 Mishandling of Exceptional Conditions



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (1/2):
 - **Fallo Seguro (Fail Closed):** Ante la duda o el error, denegar el acceso. Si el sistema de validación de tickets se cae, no dejas pasar a todo el mundo; cierras la puerta hasta que se arregle.
 - **Mensajes de usuario genéricos e inofensivos:** El usuario final solo debe ver mensajes amigables como: "Uy, algo ha salido mal por nuestra parte. Por favor, inténtalo de nuevo más tarde." (Sin códigos técnicos ni rutas).

OWASP TOP 10

A10:2025 Mishandling of Exceptional Conditions



- Los mecanismos de prevención para este tipo de ataque se pueden resumir en (2/2):
 - **Registro centralizado y detallado:** El detalle técnico (el Stack Trace) **NUNCA** va a la pantalla del usuario. Se debe enviar directa y exclusivamente a los logs internos del servidor (SIEM) para que los desarrolladores lo investiguen → Relacionado con A09:2025 Security Logging & Alerting Failures
 - **Uso Universal de Try-Catch:** Asegurarse de que el código está preparado para capturar excepciones previsibles e imprevisibles en todas las funciones críticas, evitando que el servidor colapse.

OWASP TOP 10

A10:2025 Mishandling of Exceptional Conditions



• Ejemplo – Error Aplicación Spring

Whitelabel Error Page

This application has no explicit mapping for /error, so you are seeing this as a fallback.

Mon Aug 17 11:21:04 IST 2020

There was an unexpected error (type=Internal Server Error, status=500).

Unable to execute HTTP request: dynamodb.us-east-1.amazonaws.com

```
com.amazonaws.SdkClientException: Unable to execute HTTP request: dynamodb.us-east-1.amazonaws.com
    at com.amazonaws.http.AmazonHttpClient$RequestExecutor.handleRetryableException(AmazonHttpClient.java:1113)
    at com.amazonaws.http.AmazonHttpClient$RequestExecutor.executeHelper(AmazonHttpClient.java:1063)
    at com.amazonaws.http.AmazonHttpClient$RequestExecutor.doExecute(AmazonHttpClient.java:743)
    at com.amazonaws.http.AmazonHttpClient$RequestExecutor.executeWithTimer(AmazonHttpClient.java:717)
    at com.amazonaws.http.AmazonHttpClient$RequestExecutor.execute(AmazonHttpClient.java:699)
    at com.amazonaws.http.AmazonHttpClient$RequestExecutor.access$500(AmazonHttpClient.java:667)
    at com.amazonaws.http.AmazonHttpClient$RequestExecutionBuilderImpl.execute(AmazonHttpClient.java:649)
    at com.amazonaws.http.AmazonHttpClient.execute(AmazonHttpClient.java:513)
    at com.amazonaws.services.dynamodbv2.AmazonDynamoDBClient.doInvoke(AmazonDynamoDBClient.java:2186)
    at com.amazonaws.services.dynamodbv2.AmazonDynamoDBClient.invoke(AmazonDynamoDBClient.java:2162)
```

Un usuario normal
no debe nunca ver
esta página

OWASP TOP 10

A10:2025 Mishandling of Exceptional Conditions



- Ejemplo – Error Aplicación Spring

☹️ **Something went wrong!**

Our Engineers are on it.

Go Home

Si es un error inesperado o del que no queremos dar detalles, mejor una página genérica

Seguridad Web

- Conceptos básicos de seguridad web
- OWASP TOP 10
- **Otras vulnerabilidades**
- Conclusiones

Otras vulnerabilidades

- En este tema hemos recogido parte de las vulnerabilidades clásicas que pueden afectar a una aplicación web (TOP 10 de OWASP)
- Sin embargo, no hemos cubierto todas
- Comentaremos 2 para cerrar el tema
 - Server-Side Request Forgery
 - Vulnerabilidades en LLMs

Otras vulnerabilidades

Server-Side Request Forgery

- La vulnerabilidad de SSRF se da cuando una aplicación web obtiene un recurso remoto sin validar la URL proporcionada por el usuario.
- Esto permite a un atacante forzar a la aplicación a enviar una request a un destino inesperado, incluso cuando hay un firewall, VPN o cualquier otro tipo de ACL a nivel de red.
- A medida que aumenta la funcionalidad que ofrecen algunas webs, obtener un recurso de una URL remota se convierte en un escenario habitual.

Otras vulnerabilidades

Server-Side Request Forgery

- Tenía su propia categoría en el OWASP TOP 10 (2021), pero han decidido incluirla **A01:2025 - Broken Access Control** porque se considera un fallo en el control de acceso.
 - En este curso, hemos considerado que es una vulnerabilidad con suficiente entidad como para contarse por separado



Otras vulnerabilidades

Server-Side Request Forgery

- Ejemplos de ataques (1/2)
 - **Abuso de esquemas alternativos (file://):** Si la librería que descarga la URL no está restringida, el atacante no solo usa http://. Puede inyectar file:///etc/passwd para que el servidor lea sus propios archivos de sistema y se los devuelva al atacante.
 - **Ataques a APIs Internas:** Un sistema de microservicios donde el frontend llama a `http://api-interna:5000/borrar-usuario?id=123`. El atacante usa SSRF para forzar al frontend a hacer exactamente esa llamada, la cual es aceptada porque viene de una máquina de confianza.



Otras vulnerabilidades

Server-Side Request Forgery

- Ejemplos de ataques (2/2)
 - **Ataque a metadatos en la nube:** Un conversor de PDF online te pide la URL de un documento. El atacante introduce `http://169.254.169.254/latest/meta-data/iam/security-credentials/`. El conversor descarga el archivo que contiene los tokens de acceso de Amazon Web Services y se los muestra al atacante en el PDF resultante.
 - **Escaneo de puertos y servicios locales:** El atacante prueba URLs como `http://localhost:8080` o `http://127.0.0.1:22` en una función de importación. Dependiendo del tiempo de respuesta o del error devuelto, el atacante deduce si hay un panel de administración o un servicio SSH corriendo internamente.

Otras vulnerabilidades

Server-Side Request Forgery



- Contramedidas y prevención (1/2)
 - **Listas Blancas (Allow-lists) estrictas:** Si tu aplicación solo necesita descargar avatares de `https://api.github.com`, rechaza cualquier URL que no empiece exactamente así. Evita las listas negras (block-lists), ya que los atacantes pueden evadirlas fácilmente usando acortadores de URL o IPs ofuscadas.
 - **Validación de IPs y dominios:** El servidor debe resolver el dominio introducido por el usuario y verificar matemáticamente que la IP resultante no sea una IP privada (192.168.x.x, 10.x.x.x), de loopback (127.0.0.1) o de metadatos cloud.

Otras vulnerabilidades

Server-Side Request Forgery



- Contramedidas y prevención (2/2)
 - **Desactivar esquemas peligrosos:** Configurar el cliente HTTP (la librería que hace la petición, como cURL, axios o requests) para que solo acepte http:// y https://, bloqueando file://, ftp:// o gopher://.
 - **No seguir redirecciones a ciegas:** Un atacante puede dar una URL permitida (ej. su propia web http://hacker.com) que luego responde con una redirección HTTP 302 hacia http://localhost. El servidor no debe seguir esas redirecciones sin re-validar el nuevo destino.

Otras vulnerabilidades

Vulnerabilidades en LLMs

- Los modelos grandes de lenguaje (Large-Language Models, LLMs) han irrumpido cambiando la forma en la que trabajamos
- Cada vez más encontramos aplicaciones web que utilizan LLMs internamente
 - ChatBots
 - Traductores
 - Recomendadores
 - Y un largo etc

Otras vulnerabilidades

Vulnerabilidades en LLMs

- OWASP ha publicado un TOP 10 exclusivo de LLMs:
Top 10 Risk & Mitigations for LLMs and Gen AI Apps

<https://genai.owasp.org/llm-top-10/>

- Explora vulnerabilidades y contramedidas para trabajar con IA generativa (LLMs)

LLM01: 2025

Prompt Injection

LLM01:2025 Prompt Injection

A Prompt Injection Vulnerability occurs when user prompts alter the...

[Read More](#)

LLM02: 2025

Sensitive Information Disclosure

LLM02:2025 Sensitive Information Disclosure

Sensitive information can affect both the LLM and its application...

[Read More](#)

LLM03: 2025

Supply Chain

LLM03:2025 Supply Chain

LLM supply chains are susceptible to various vulnerabilities, which can...

[Read More](#)

LLM04: 2025

Data and Model Poisoning

LLM04:2025 Data and Model Poisoning

Data poisoning occurs when pre-training, fine-tuning, or embedding data is...

[Read More](#)

LLM05: 2025

Improper Output Handling

LLM05:2025 Improper Output Handling

Improper Output Handling refers specifically to insufficient validation, sanitization, and...

[Read More](#)

LLM06: 2025

Excessive Agency

LLM06:2025 Excessive Agency

An LLM-based system is often granted a degree of agency...

[Read More](#)

LLM07: 2025

System Prompt Leakage

LLM07:2025 System Prompt Leakage

The system prompt leakage vulnerability in LLMs refers to the...

[Read More](#)

LLM08: 2025

Vector and Embedding Weaknesses

LLM08:2025 Vector and Embedding Weaknesses

Vectors and embeddings vulnerabilities present significant security risks in systems...

[Read More](#)

LLM09: 2025

Misinformation

LLM09:2025 Misinformation

Misinformation from LLMs poses a core vulnerability for applications relying...

[Read More](#)

LLM10: 2025

Unbounded Consumption

LLM10:2025 Unbounded Consumption

Unbounded Consumption refers to the process where a Large Language...

[Read More](#)

Seguridad Web

- Conceptos básicos de seguridad web
- OWASP TOP 10
- Otras vulnerabilidades
- **Conclusiones**

Conclusiones

- Hemos realizado un repaso al top 10 de OWASP tomando un **enfoque defensivo**
- Es importante tener en cuenta que, la mayoría de las veces, un ataque no se apoya en una sola vulnerabilidad, sino que la concatenación de varias vulnerabilidades permite materializar las amenazas (hemos visto algunos ejemplos)

Conclusiones

- En definitiva, existen un sinnúmero de vulnerabilidades que pueden estar, potencialmente, presentes en una aplicación web.
- Se deben **implementar mecanismos que nos permitan detectar y parchear estas vulnerabilidades** antes de que se materialicen en amenazas.
 - Análisis de código (estático, dinámico, interactivo).
 - Un buen pipeline de CD/CI.
 - Buena batería de pruebas (QA end-to-end).

Conclusiones

- Podéis encontrar más ejemplos de vulnerabilidades y cómo explotarlas en

<https://github.com/swisskyrepo/PayloadsAllTheThings/tree/master>

- Para mitigarlas y evitar introducirlas en vuestro código, lo mejor es seguir las recomendaciones de los frameworks de desarrollo, así como la guía de OWASP.